# Elasticity-Only Modeling for Randomized Motor-Insurance Price Tests

## A detailed tutorial on Bayes-flip identification, constrained models, tree ensembles, Optuna tuning, calibration, and real-data validation

Research tutorial

August 2026

## Table of Contents



*The table lists major chapters. Detailed subsections appear as numbered headings throughout the tutorial.*

# Purpose and scope

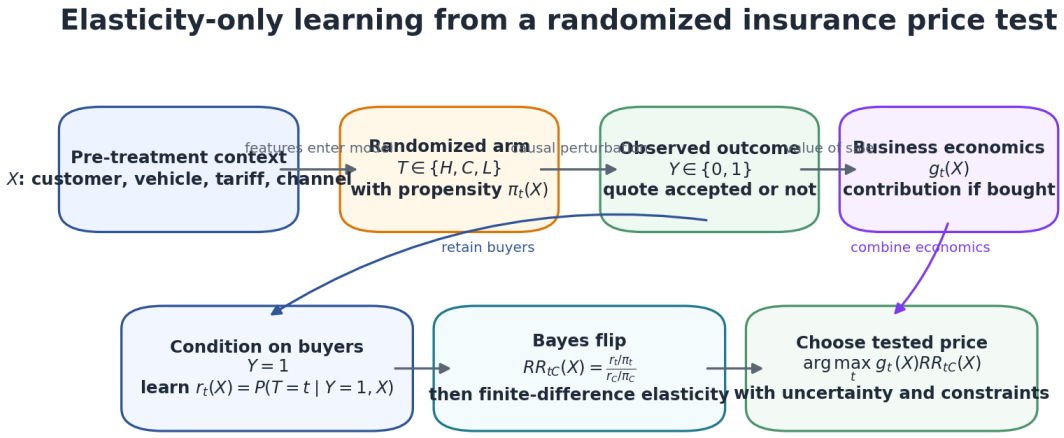
This tutorial explains how to estimate **price response alone** from a randomized motor-insurance price test in which eligible quotes are assigned to a high-price arm, a control arm, or a low-price arm. The central modeling idea is to condition on customers who bought the policy and reverse the usual prediction direction:

- the usual direction predicts purchase from price arm and customer context;
- the **Bayes-flip** direction predicts price arm from customer context among buyers;
- Bayes' rule then converts the buyer-arm probabilities into conversion **risk ratios**, from which standard relative-change, logarithmic, midpoint, or point-elasticity summaries can be derived.

The tutorial is deliberately self-contained. Every symbol is defined before it is used, and the statistical, machine-learning, and business terms are explained rather than assumed.

The intended audience is a data scientist, actuary, machine-learning engineer, or LLM agent working with **real randomized quote-level insurance data**. The document does not claim that any model family is empirically best before it has been compared on the insurer's own experiment. It contains mathematical derivations, model designs, Python patterns, validation rules, Optuna search spaces, and production-quality failure checks. It contains no synthetic leaderboard.

> **Central recommendation.** Treat global elasticity as the mandatory anchor. Use exact pairwise buyer likelihoods as the primary flexible formulation. Compare penalized logistic regression, LightGBM, XGBoost, and a strongly smoothed Random Forest under the same nested, group-disjoint, time-aware validation design. Calibrate and shrink every flexible model out of fold. Deploy personalized pricing only when the flexible model beats the global anchor in proper likelihood, randomized effect calibration, stability, and business value.




*All quotes remain essential for experiment auditing and unbiased policy evaluation, even when the primary response model is trained on buyers.*

*End-to-end map of the elasticity-only problem*

# 1. The business problem

## 1.1 What is being priced?

The product is mandatory motor insurance, but the relevant outcome is not whether the customer obtains insurance somewhere. The relevant outcome is whether the customer accepts **this insurer's quote**. A customer who rejects the quote may buy from a competitor, abandon the shopping process, delay the decision, switch channel, or obtain a revised quote.

The experiment therefore measures the response of **own-quote conversion** to a randomized price perturbation. It does not directly measure total market demand for legally required insurance.

## 1.2 The three experimental arms

Each eligible quote is randomly assigned to one of three arms:

- **High arm**, denoted by $H$: approximately $+10\%$ relative to the normal quote;
- **Control arm**, denoted by $C$: the normal quote;
- **Low arm**, denoted by $L$: approximately $-10\%$ relative to the normal quote.

We write the randomized arm for quote $i$ as

$$T_i \in \{H, C, L\}.$$

The subscript $i$ indexes a quote opportunity. It does not necessarily index a unique person: one customer may create several quotes. This is why customer or household identifiers matter in validation.

The purchase outcome is

$$Y_i = \begin{cases} 1, & \text{if the quote is accepted within the specified outcome window,} \\ 0, & \text{otherwise.} \end{cases}$$

The vector of pre-treatment information is denoted by $X_i$. It may contain customer, driver, vehicle, coverage, channel, tariff, market, and time variables that were known **before** the randomized price was shown.

## 1.3 What the business wants to know

The main objective is not to predict the absolute probability of purchase. It is to identify contexts in which:

- a price increase is likely to reduce conversion by little enough that contribution rises;
- a price reduction is likely to increase conversion enough to justify the discount;
- neither adjustment is sufficiently attractive or sufficiently certain, so control should be retained.

The core statistical outputs are therefore the relative conversion risks

$$RR_{HC}(x) = \frac{q_H(x)}{q_C(x)}, RR_{LC}(x) = \frac{q_L(x)}{q_C(x)},$$

where $x$ denotes a particular customer and quote context, and

$$q_t(x) = P(Y = 1 \mid T = t, X = x)$$

is the conversion probability under arm $t$.

The notation $RR$ means **risk ratio**. Here the word *risk* is statistical terminology for a probability. It does not mean insurance claim risk.

## 1.4 Why absolute conversion can cancel from the price choice

Let

$$g_t(x)$$

be the insurer's contribution **conditional on purchase** under arm $t$. Depending on the business definition, it can include:

- premium;
- expected claims;
- reinsurance and capital charges;
- commissions and taxes;
- payment and acquisition costs that are incurred only when the policy is sold;
- expected renewal or customer-lifetime value, if the decision is genuinely optimized on that horizon.

The expected contribution per quote under arm $t$ is

$$V_t(x) = q_t(x) g_t(x).$$

Because $q_t(x) = q_C(x) RR_{tC}(x)$,

$$V_t(x) = q_C(x) RR_{tC}(x) g_t(x).$$

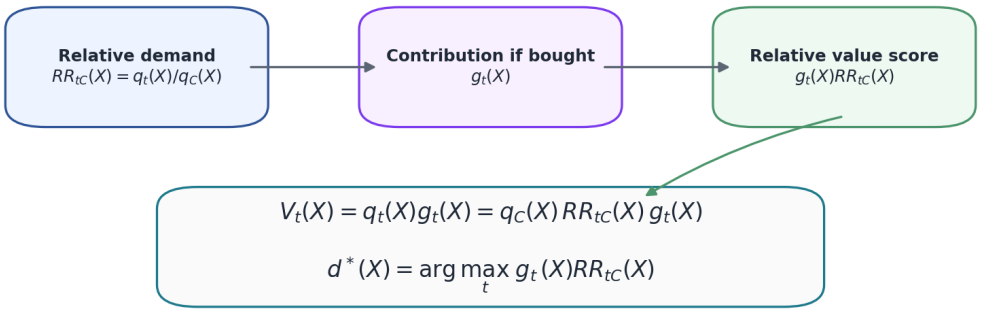
For a fixed quote context $x$, the positive factor $q_C(x)$ is common to all three candidate prices. Therefore,

$$d^{\star}(x)=arg \max_{t \in \{H,C,L\}} g_t(x) R R_{tC}(x),$$

with $R R_{CC}(x)=1.$

The symbol $d^{\star}(x)$ denotes the price decision selected for context $x$. The star indicates the best decision under the stated objective and model.

**Why relative demand is sufficient for the tested-price choice**




*The cancellation fails when the action creates quote-level costs or constraints that are not multiplied by purchase probability, or when the objective includes capacity, fairness, retention, or lifetime-value terms not represented in $g_t$.*

*Relative demand and contribution determine the tested-price choice*

### When this cancellation does not apply

The cancellation is exact only when the objective has the form $q_t(x) g_t(x)$. A base-conversion model may become relevant when the decision also includes terms that are not multiplied by purchase probability. Examples include:

- a quote-level operational cost that depends on the selected arm even when no purchase occurs;
- a hard sales-volume, market-share, or capacity constraint;
- a regulatory or fairness constraint imposed at portfolio level;
- a customer-contact or retention consequence that occurs before purchase;
- an optimization target that combines current contribution with a separately modeled future outcome;
- a requirement to forecast absolute sales volume, cash flow, or solvency capital.

The elasticity-only approach should therefore be interpreted as a clean solution to the **relative price-choice component**, not as a universal replacement for every business model.

## 2. Experimental and causal foundations

### 2.1 Potential outcomes

A useful way to define the causal target is through potential outcomes. For each eligible quote $i$, imagine three binary outcomes:

$$Y_i(H), Y_i(C), Y_i(L).$$

$Y_i(H)$ is the purchase outcome that would occur if quote $i$ were assigned to the high arm; $Y_i(C)$ and $Y_i(L)$ are defined analogously.

Only one of these outcomes is observed, because the quote receives only one randomized arm. The observed outcome satisfies the **consistency** relation

$$Y_i = Y_i(T_i).$$

Consistency means that the arm label corresponds to a well-defined intervention and that the observed outcome under the assigned arm is the corresponding potential outcome.

## 2.2 Randomization propensity

The probability that quote $i$ is assigned to arm $t$ is called the **assignment propensity**, **randomization probability**, or **logging probability**:

$$\pi_{i,t} = P\left(T_i = t \mid X_i, \text{randomization block information}\right).$$

When the design is fixed at $10\,\%/80\,\%/10\,\%$ for every eligible quote,

$$\pi_H = 0.1, \pi_C = 0.8, \pi_L = 0.1.$$

In production experiments, the propensities may vary by date, quota, channel, risk block, or operational constraint. The row-level logged values should then be used. A nominal overall allocation is not a substitute for the propensity that actually governed a particular quote.

The propensities must satisfy

$$0 < \pi_{i,t} < 1, \pi_{i,H} + \pi_{i,C} + \pi_{i,L} = 1.$$

The requirement that every relevant arm has positive probability is called **positivity** or **overlap**.

## 2.3 Why randomization identifies the arm-specific risks

A valid randomized design makes the arm independent of the potential outcomes, conditional on the variables used by the randomizer:

$$T_i \perp \{Y_i(H), Y_i(C), Y_i(L)\} \mid X_i.$$

The symbol $\perp$ means statistical independence.

Under randomization, consistency, positivity, and a stable outcome definition,

$$q_t(x) = P(Y = 1 \mid T = t, X = x)$$

has a causal interpretation as the conversion probability that would be observed under arm $t$ for contexts like $x$.

This protection is important in insurance. The normal technical price is estimated from customer and vehicle history and may contain substantial error. Randomization prevents that price error from confounding the **assigned multiplier effect**, although it can still make effect heterogeneity difficult to learn.

## 2.4 Intention-to-treat and realized price

The primary causal treatment should normally be the randomized assignment $T_i$. This is the **intention-to-treat** estimand.

The displayed premium may differ from the nominal multiplier because of:

- rounding;
- minimum premiums;
- taxes and fees;
- installments;
- manual intervention;
- underwriting referral;
- a technical or user-interface failure.

Let $P_{i,t}$ denote the premium that would be displayed under arm $t$, and let $P_{i,T_i}$ be the observed displayed premium. If assignment and realized dose differ, replacing the randomized arm with the observed percentage change can reintroduce confounding. Realized-dose analysis should be treated as a compliance or instrumental-variable problem, not as an automatic substitute for intention-to-treat.

## 2.5 No interference and repeat quotes

A standard assumption is that one quote's randomized arm does not change another quote's outcome. This is a form of **no interference**.

In practice, a customer may request several quotes and may remember earlier prices. The independence assumption can therefore fail across repeated opportunities. At minimum:

- repeated quotes must be linked by a stable customer or household identifier;
- the experimental protocol should specify how repeat quotes are randomized;
- validation folds must keep all quotes from the same customer or household together;
- sensitivity analyses should examine first quote only, first randomized episode, or a washout rule.

# 3. Probability and modeling notation

This chapter defines the mathematical tools used throughout the tutorial.

## 3.1 Conditional probability

$P(A \mid B)$ means the probability of event $A$ given that event or information $B$ is known.

For example,

$$P(Y=1 \mid T=H, X=x)$$

is the purchase probability among quotes with context $x$ that receive the high-price arm.

## 3.2 Risk and risk ratio

For a binary outcome, a **risk** is simply a probability. The risk ratio of arm $t$ relative to control is

$$RR_{tC}(x)=\frac{q_t(x)}{q_C(x)}.$$

Interpretations:

- $RR_{HC}(x)=0.90$ means the high arm has $90\,\%$ of the control conversion probability, a relative reduction of $10\,\%$;
- $RR_{LC}(x)=1.25$ means the low arm has $125\,\%$ of the control conversion probability, a relative increase of $25\,\%$.

A risk ratio is not an **odds ratio**.

## 3.3 Odds and logit

For a probability $p \in (0,1)$, the odds are

$$odds(p)=\frac{p}{1-p}.$$

The **logit** is the logarithm of the odds:

$$logit(p)=\log\frac{p}{1-p}.$$

The inverse-logit or logistic function is

$$\sigma(a)=\frac{1}{1+e^{-a}}.$$

Thus,

$$\sigma(logit(p))=p.$$

Logistic regression models a log-odds quantity. In the pairwise Bayes-flip formulation developed later, the modeled log-odds separates into a known propensity offset plus a **log risk ratio**. This is why a logistic classifier can estimate relative conversion risk exactly in this special conditional problem.

## 3.4 Log risk ratio

The logarithm of a risk ratio is

$$\ell_{tC}(x)=\log RR_{tC}(x).$$

This tutorial often calls it `log_rr` in Python code.

The log scale is convenient because:

- no effect corresponds to $\ell_{tC}=0$;
- a multiplicative risk ratio becomes additive;
- monotonicity is a sign constraint:

$$\ell_{HC}\le 0,\ell_{LC}\ge 0;$$

- model ensembles can average log risk ratios without producing negative risks.

### 3.5 Softmax

For three real-valued scores $a_H, a_C, a_L$, the softmax function produces probabilities that are positive and sum to one:

$$softmax(a)_t=\frac{e^{a_t}}{e^{a_H}+e^{a_C}+e^{a_L}}.$$

Softmax is the multiclass analogue of the logistic function. It is used to reconstruct a coherent three-arm buyer posterior from two risk-ratio predictions.

### 3.6 Cross-entropy and negative log-likelihood

Suppose a model assigns probability $p_i$ to the class that actually occurs for observation $i$. Its log loss is

$$-\log p_i.$$

The average is called **cross-entropy** or **negative log-likelihood** (NLL) in this setting.

A lower NLL is better. Log loss is a **strictly proper scoring rule**: in expectation, it is minimized by the true conditional probabilities. This matters because elasticity depends on the numerical probability ratios, not merely the ordering of customers.

### 3.7 Offset

An **offset** is a known additive term in a linear predictor whose coefficient is fixed at one.

If

$$logit(p_i)=o_i+f(X_i),$$

then $o_i$ is the offset and $f$ is learned. In the pairwise buyer model,

$$o_i=\log\frac{\pi_{i,t}}{\pi_{i,C}}$$

is determined by the randomized design. The model should not waste capacity estimating it.

### 3.8 Sample weights and inverse-propensity weights

A sample weight changes how much an observation contributes to the training loss. An **inverse-propensity weight** for a quote assigned to arm $t$ is proportional to

$$\frac{1}{\pi_{i,t}}.$$

Such weighting can transform the buyer-arm distribution into a normalized demand distribution. It is useful when a learning library cannot accept a fixed offset, but it can increase variance and must not be combined casually with class balancing.

## 4. The Bayes-flip identity

### 4.1 Forward and flipped probabilities

The forward conversion risk is

$$q_t(x)=P(Y=1\mid T=t,X=x).$$

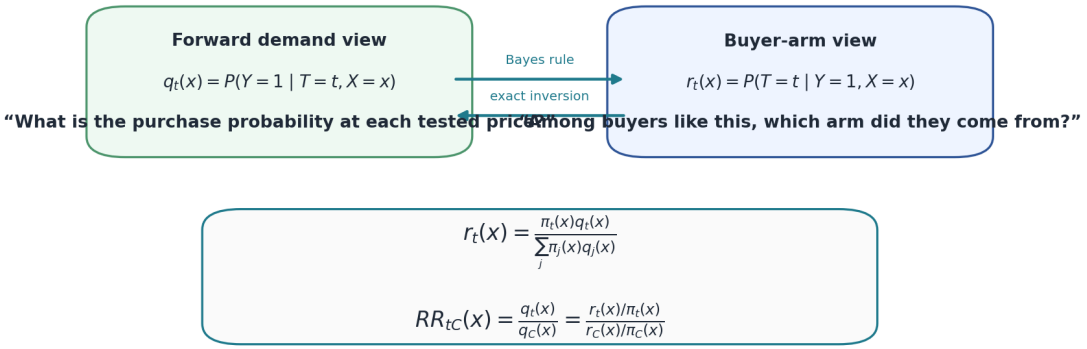
The flipped buyer-arm probability is

$$r_t(x) = P(T = t \mid Y = 1, X = x).$$

$r_t(x)$ answers a different-looking question: among buyers with context $x$, what is the probability that the buyer came from arm $t$?

The word **posterior** is sometimes used for $r_t(x)$ because it is a probability of the arm after observing purchase. The term does not imply that the model must use Bayesian priors or posterior sampling.

**The Bayes flip reverses the prediction direction without changing the identified risk ratios**




*The nickname "Bayes-flip" refers to Bayes' rule. The estimator itself may be frequentist, Bayesian, or machine-learning based.*

*Forward and flipped probability views*

## 4.2 Step-by-step derivation

Bayes' rule states

$$P(T = t \mid Y = 1, X = x) = \frac{P(Y = 1 \mid T = t, X = x) P(T = t \mid X = x)}{P(Y = 1 \mid X = x)}.$$

Using the notation above,

$$r_t(x) = \frac{q_t(x) \pi_t(x)}{P(Y = 1 \mid X = x)}.$$

The marginal purchase probability in the denominator is found by summing over the three randomized arms:

$$P(Y = 1 \mid X = x) = \sum_{j \in \{H, C, L\}} \pi_j(x) q_j(x).$$

Therefore,

$$r_t(x) = \frac{\pi_t(x) q_t(x)}{\sum_j \pi_j(x) q_j(x)}.$$

Divide the expression for arm $t$ by the expression for control:

$$\frac{r_t(x)}{r_C(x)} = \frac{\pi_t(x) q_t(x)}{\pi_C(x) q_C(x)}.$$

Rearranging gives the central identification formula:

$$RR_{tC}(x) = \frac{q_t(x)}{q_C(x)} = \frac{r_t(x)/\pi_t(x)}{r_C(x)/\pi_C(x)}.$$

This is exact. It does not require conversion to be rare.

Case-only methods for randomized trials use a closely related argument to identify treatment-effect modification on the relative-risk scale [1]. The insurance application differs in subject matter, but the probability identity is the same.

## 4.3 The fixed 10/80/10 experiment

For

$$(\pi_H, \pi_C, \pi_L) = (0.1, 0.8, 0.1),$$

we obtain

$$RR_{HC}(x) = \frac{r_H(x)/0.1}{r_C(x)/0.8} = \frac{8\, r_H(x)}{r_C(x)},$$

and

$$RR_{LC}(x) = \frac{r_L(x)/0.1}{r_C(x)/0.8} = \frac{8\, r_L(x)}{r_C(x)}.$$

The factor eight appears because control is eight times as likely to be assigned as either side arm.

## 4.4 Monotonicity constraint

Economically, increasing price should not increase purchase probability, and decreasing price should not reduce it. The weak monotonicity condition is

$$q_H(x) \le q_C(x) \le q_L(x).$$

On the risk-ratio scale,

$$RR_{HC}(x) \le 1 \le RR_{LC}(x).$$

Using the Bayes-flip identity, the general propensity-adjusted form is

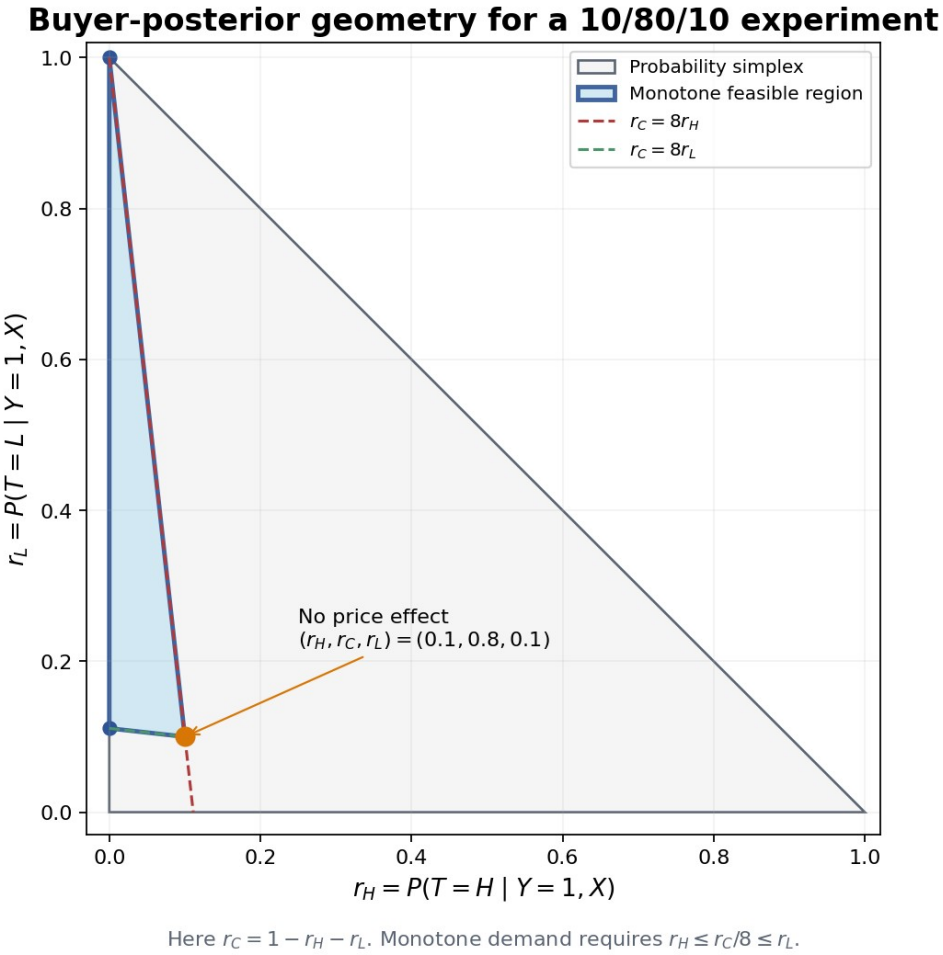
$$\frac{r_H(x)}{\pi_H(x)} \le \frac{r_C(x)}{\pi_C(x)} \le \frac{r_L(x)}{\pi_L(x)}.$$

For the fixed $10/80/10$ design,

$$r_H(x) \le \frac{r_C(x)}{8} \le r_L(x).$$

The inequalities should normally be weak. No price effect is a legitimate possibility and lies on the boundary:

$$r(x) = (0.1, 0.8, 0.1).$$




*The constrained buyer-posterior triangle*

## 4.5 A numerical illustration

The following numbers are a hand calculation, not empirical evidence. Suppose a fitted model gives

$$(r_H, r_C, r_L) = (0.075, 0.790, 0.135).$$

Then

$$RR_{HC} = \frac{8(0.075)}{0.790} \approx 0.759,$$

and

$$RR_{LC} = \frac{8(0.135)}{0.790} \approx 1.367.$$

The interpretation is that, for contexts like this one, conversion under the high arm is estimated at about $75.9\,\%$ of control conversion, while conversion under the low arm is estimated at about $136.7\,\%$ of control conversion.

## 4.6 What buyer-only data do not identify

Multiplying all three conversion risks by the same positive factor leaves $r_t(x)$ unchanged. For example,

$$(q_H, q_C, q_L) = (0.03, 0.04, 0.055)$$

and

$$(q_H, q_C, q_L) = (0.06, 0.08, 0.11)$$

have identical risk ratios and therefore identical buyer-arm posteriors under the same propensities.

The buyer likelihood identifies relative demand but not the absolute conversion scale. This is acceptable for the independent tested-price choice under the conditions in Section 1.4. It is not sufficient for absolute sales forecasting.

### 4.7 Why conditioning on buyers can help in practice

Conditioning does not create information. A correctly specified complete likelihood uses buyer and nonbuyer information and is at least as informative in principle.

The practical advantage is **nuisance removal**. With roughly $5\%$ conversion, a generic all-quote binary loss can devote most of its capacity to predicting baseline purchase probability. That baseline may be difficult because of tariff estimation error, unobserved competitor position, shopping intent, and other factors. The buyer likelihood removes the common conversion scale and places the optimization pressure on randomized arm composition among purchasers.

This is a robustness argument under model misspecification, not a theorem that buyers contain more information than the full cohort.

## 5. From risk ratios to price elasticity

### 5.1 Why several formulas are called “elasticity”

The word **elasticity** is used for several closely related quantities. Confusion arises when a point derivative and a finite experimental contrast are written with the same symbol.

For one comparison, let:

- $P_0>0$ be the reference price, usually the control price;
- $P_1>0$ be the alternative price, such as the high or low tested price;
- $q_0=q(P_0,x)>0$ be conversion at the reference price for context $x$;
- $q_1=q(P_1,x)>0$ be conversion at the alternative price;
- $m=P_1/P_0$ be the **price ratio**;
- $R=q_1/q_0$ be the conversion **risk ratio**.

The experiment and the Bayes-flip model identify $R$. An elasticity is then obtained by combining $R$ with the known price ratio $m$.

Four conventions are useful:

1. **base-period relative-change elasticity**, often used in business reporting;
2. **point elasticity**, defined by a derivative;
3. **log-change finite-difference elasticity**, used as the default structural quantity in this tutorial;
4. **midpoint arc elasticity**, a symmetric finite-change convention.

All four have the same infinitesimal limit. For a finite $10\%$ change, however, their numerical values need not be equal.

### 5.2 Standard base-period relative-change elasticity

The finite-change definition commonly used in business is

$$\varepsilon_{0\to1}^{rel}(x)=\frac{(q_1-q_0)/q_0}{(P_1-P_0)/P_0}.$$

The numerator is the conversion change relative to the reference conversion. The denominator is the price change relative to the reference price.

Using $R=q_1/q_0$ and $m=P_1/P_0$ gives the exact and convenient form

$$\varepsilon_{0\to1}^{rel}(x)=\frac{R(x)-1}{m-1}.$$

For decreasing demand this signed elasticity is negative. For example, if a $10\%$ price increase produces a $20\%$ relative conversion decrease, then

$$\varepsilon^{rel}=\frac{-0.20}{0.10}=-2.$$

For the three-arm experiment,

$$\varepsilon_{HC}^{rel}(x)=\frac{RR_{HC}(x)-1}{P_H(x)/P_C(x)-1},$$

and

$$\varepsilon_{LC}^{rel}(x)=\frac{RR_{LC}(x)-1}{P_L(x)/P_C(x)-1}.$$

With nominal multipliers $1.10$ and $0.90$,

$$\varepsilon_{HC}^{rel}(x)=\frac{RR_{HC}(x)-1}{0.10},$$

$$\varepsilon_{LC}^{rel}(x)=\frac{RR_{LC}(x)-1}{-0.10}.$$

Both values are negative under monotone demand. If the organization reports a positive sensitivity magnitude, define

$$\beta_{HC}^{rel}(x)=-\varepsilon_{HC}^{rel}(x)=\frac{1-RR_{HC}(x)}{0.10},$$

and

$$\beta_{LC}^{rel}(x)=-\varepsilon_{LC}^{rel}(x)=\frac{RR_{LC}(x)-1}{0.10}.$$

The arrow $0 \to 1$ matters. Base-period percentage changes are directional because the old value appears in the denominator. Reversing the comparison generally changes the numerical elasticity:

$$\varepsilon_{1\to 0}^{rel}=\frac{m}{R}\varepsilon_{0\to 1}^{rel}.$$

This direction dependence is not an error; it is a property of the chosen percentage-change convention.

## 5.3 Point elasticity and its connection to relative changes

If the demand curve $q(p,x)$ is differentiable in price, the point price elasticity is

$$\varepsilon(p,x)=\frac{p}{q(p,x)}\frac{\partial q(p,x)}{\partial p}.$$

Using the chain rule,

$$\frac{\partial \log q(p,x)}{\partial \log p}=\frac{\partial \log q}{\partial q}\frac{\partial q}{\partial p}\frac{\partial p}{\partial \log p}=\frac{1}{q}\frac{\partial q}{\partial p}p.$$

Therefore,

$$\varepsilon(p,x)=\frac{\partial \log q(p,x)}{\partial \log p}.$$

The log-derivative expression is not a different definition from the usual derivative form. It is exactly the same point elasticity written in dimensionless coordinates.

The standard finite relative-change elasticity converges to point elasticity as the tested price approaches the reference price:

$$\frac{\lim_{P_1\to P_0}(q_1-q_0)/q_0}{(P_1-P_0)/P_0}=\frac{P_0}{q_0}\frac{\partial q(P_0,x)}{\partial p}.$$

Thus the ordinary percentage-change formula is a finite-difference approximation to point elasticity when the price interval is small.

## 5.4 Log-change finite-difference elasticity

With only three tested prices, the derivative at a point is not observed without a structural assumption. A particularly useful finite contrast is

$$\varepsilon_{0,1}^{\log}(x)=\frac{\log q_1(x)-\log q_0(x)}{\log P_1(x)-\log P_0(x)}=\frac{\log R(x)}{\log m}.$$

This is a **secant slope in the log-demand versus log-price plane**. It should be called a log-change finite-difference elasticity or log-arc elasticity, not silently described as the point derivative.

It also has an exact average-derivative interpretation. Let $u=\log p$. Because

$$\frac{d\log q(e^u,x)}{du}=\varepsilon(e^u,x),$$

we have

$$\log q(P_1,x)-\log q(P_0,x)=\int_{\log P_0}^{\log P_1}\varepsilon(e^u,x)\,du.$$

Consequently,

$$\varepsilon_{0,1}^{\log}(x)=\frac{1}{\log(P_1/P_0)}\int_{\log P_0}^{\log P_1}\varepsilon(e^u,x)\,du.$$

The log-change elasticity is therefore the average point elasticity over the interval when averaging uniformly in log price.

For the experiment,

$$\varepsilon_{HC}^{\log}(x)=\frac{\log RR_{HC}(x)}{\log(P_H/P_C)},$$

$$\varepsilon_{LC}^{\log}(x)=\frac{\log RR_{LC}(x)}{\log(P_L/P_C)}.$$

It is often convenient to report positive magnitudes

$$\beta_{HC}^{\log}(x)=-\varepsilon_{HC}^{\log}(x),\beta_{LC}^{\log}(x)=-\varepsilon_{LC}^{\log}(x).$$

For nominal multipliers,

$$\beta_{HC}^{\log}(x)=-\frac{\log RR_{HC}(x)}{\log 1.10},$$

and

$$\beta_{LC}^{\log}(x)=-\frac{\log RR_{LC}(x)}{\log 0.90}.$$

Because $\log 0.90<0$, the second magnitude is positive when $RR_{LC}\geq 1$.

## 5.5 Exact relationship between the standard and logarithmic formulas

The two finite-change conventions are deterministic transformations of the same pair $(R,m)$. Since

$$R=1+\varepsilon^{rel}(m-1),$$

we obtain

$$\varepsilon^{\log}=\frac{\log\left[1+\varepsilon^{rel}(m-1)\right)}{\log m}.$$

Conversely, because $R=m^{\varepsilon^{\log}}$,

$$\varepsilon^{rel}=\frac{m^{\varepsilon^{\log}}-1}{m-1}.$$

The first transformation requires

$$1+\varepsilon^{rel}(m-1)>0,$$

which is exactly the requirement that the implied risk ratio be positive.

For small relative changes, let

$$\delta_p=m-1,\delta_q=R-1.$$

Then

$$\varepsilon^{rel}=\frac{\delta_q}{\delta_p},$$

whereas

$$\varepsilon^{\log}=\frac{\log(1+\delta_q)}{\log(1+\delta_p)}.$$

Using $\log(1+z)=z-z^2/2+O(z^3)$ gives

$$\varepsilon^{\log}=\varepsilon^{rel}\left[1+\frac{\delta_p-\delta_q}{2}+O(\delta_p^2+\delta_q^2)\right).$$

Hence the definitions are nearly equal only when both the price change and the induced conversion change are small. A $10\,\%$ price change can still produce a much larger conversion change, so the difference may be material in this application.

## 5.6 Worked example: one constant elasticity, different standard finite elasticities

Suppose the true demand curve has constant point elasticity $-3$:

$$q(p,x)=a(x)\,p^{-3}.$$

Then the risk ratio for a price multiplier $m$ is

$$R=m^{-3}.$$

The resulting quantities are:

| Contrast | Price ratio $m$ | Risk ratio $R$ | Standard $\varepsilon^{rel}$ | Log-change $\varepsilon^{\log}$ |
|---|---|---|---|---|
| High versus control | 1.10 | 0.7513 | -2.4869 | -3.0000 |
| Low versus control | 0.90 | 1.3717 | -3.7174 | -3.0000 |

The underlying curve has the same point elasticity everywhere, and the log-change formula recovers that value exactly on both intervals. The standard base-period formula produces two different finite elasticities because it measures arithmetic percentage changes from control and because the demand response compounds nonlinearly.

This does not make the standard definition wrong. It means that **"one common standard finite elasticity on both sides" is a different structural assumption from "one common point or log elasticity on both sides."**

## 5.7 Midpoint arc elasticity

A common symmetric alternative uses midpoint denominators:

$$\varepsilon_{0,1}^{mid}=\frac{(q_1-q_0)/\left[(q_1+q_0)/2\right)}{(P_1-P_0)/\left[(P_1+P_0)/2\right)}.$$

In terms of the risk ratio and price ratio,

$$\varepsilon_{0,1}^{mid} = \frac{R-1}{R+1}\frac{m+1}{m-1}.$$

Unlike the base-period formula, midpoint elasticity is unchanged when the two endpoints are exchanged. It is often called **arc elasticity** or the **midpoint method**.

The midpoint formula closely approximates log-change elasticity because

$$\log z \approx \frac{2(z-1)}{z+1}$$

for $z$ near one. The logarithmic formula retains exact additive and constant-elasticity properties; the midpoint formula can be easier to explain to stakeholders who prefer ordinary percentage changes.

## 5.8 Advantages and disadvantages of each convention

| Quantity | Main advantage | Main limitation | Recommended role |
|---|---|---|---|
| Standard base-period relative change | Direct business statement: relative conversion change divided by relative price change | Depends on which endpoint is the reference; high- and low-side values are not directly symmetric | Required business reporting when this is the organizational convention |
| Point elasticity | Canonical local economic sensitivity; exact derivative | Not directly identified from three separated prices without structure or smoothing | Structural interpretation near control |
| Log-change finite difference | Symmetric under endpoint reversal; exact for constant-elasticity power laws; additive in log risk ratios; natural for the Bayes-flip parameterization | Less familiar; becomes infinite if one modeled conversion risk is exactly zero | Default modeling and comparison scale |
| Midpoint arc elasticity | Symmetric and based on ordinary finite changes | Lacks the exact linear log-risk-ratio structure and exact constant-elasticity property | Optional stakeholder-facing symmetric summary |

The most important practical point is that the price decision depends on $RR_{tC}$, not on which elasticity convention is printed in a report. If two analysts start from the same risk-ratio predictions and the same tested prices, they make the same arm decision even if one reports standard elasticity and the other reports log-change elasticity.

## 5.9 Modeling scale versus reporting scale

The natural Bayes-flip likelihood is a likelihood for the randomized arm among buyers. It identifies the buyer probabilities and therefore the risk ratios. It does **not** require an observed individual elasticity label.

For that reason, the recommended workflow is:

1. fit the model using buyer categorical or pairwise cross-entropy;
2. recover $RR_{HC}(x)$ and $RR_{LC}(x)$;
3. retain risk ratios as the primary decision quantities;
4. transform the same predictions into the elasticity convention required for reporting.

Changing from log-change elasticity to standard elasticity does not require changing cross-entropy into mean-squared error. The distribution of the observed arm has not changed. Only the deterministic presentation of the estimated risk ratio has changed.

For row-specific planned price ratios $m_{i,t} = P_{i,t}/P_{i,C}$, compute

$$\hat{\varepsilon}_{i,tC}^{rel} = \frac{\widehat{RR}_{i,tC} - 1}{m_{i,t} - 1},$$

and

$$\hat{\varepsilon}_{i,tC}^{\log} = \frac{\log \widehat{RR}_{i,tC}}{\log m_{i,t}}.$$

Never substitute a nominal $10\,\%$ denominator when the planned counterfactual dose for that row is materially different.

## 5.10 What to do when the standard definition is mandatory

When governance, actuarial practice, or business reporting requires the standard quotient of relative changes, use the following procedure.

### Step 1: estimate risk ratios with the proper buyer likelihood

Estimate $RR_{HC}(x)$ and $RR_{LC}(x)$ using the methods in later chapters. Do not construct noisy per-customer conversion ratios; no individual customer is observed under both prices.

### Step 2: convert each contrast separately

For nominal $\pm 10\,\%$ tests,

$$\hat{\varepsilon}_{HC}^{rel}(x) = 10\left[\widehat{RR}_{HC}(x) - 1\right],$$
$$\hat{\varepsilon}_{LC}^{rel}(x) = -10\left[\widehat{RR}_{LC}(x) - 1\right].$$

If positive magnitudes are required,

$$\hat{\beta}_{HC}^{rel}(x) = 10\left[1 - \widehat{RR}_{HC}(x)\right],$$
$$\hat{\beta}_{LC}^{rel}(x) = 10\left[\widehat{RR}_{LC}(x) - 1\right].$$

### Step 3: report the reference direction explicitly

Use labels such as:

- “control-to-high base-period elasticity”;
- “control-to-low base-period elasticity.”

Do not report a direction-free number when the formula is direction-dependent.

### Step 4: preserve both sides

Even when a scalar log-elasticity model predicts one $\beta^{\log}(x)$, the implied standard finite elasticities are generally different:

$$\varepsilon_{HC}^{rel}(x) = \frac{1.10^{-\beta^{\log}(x)} - 1}{0.10},$$
$$\varepsilon_{LC}^{rel}(x) = \frac{0.90^{-\beta^{\log}(x)} - 1}{-0.10}.$$

Therefore, output two standard finite elasticities rather than pretending that the scalar log parameter is numerically identical to both.

### Step 5: propagate uncertainty through the transformation

In a clustered bootstrap or posterior simulation, transform every replicated risk ratio into the required elasticity. Then take quantiles of the transformed values. This automatically handles nonlinearity and the reversed denominator in the low-price contrast.

### Step 6: continue to use risk ratios for the price decision

The arm-selection score remains

$$g_t(x)\,RR_{tC}(x).$$

Elasticity conventions are reporting and structural-model choices; they should not introduce a different business decision when the underlying risk ratios are unchanged.

## 5.11 Two alternative one-number structural models

A one-number model can be defined in two different ways. They should be compared empirically rather than conflated.

### Log-constant-elasticity model

Predict one nonnegative magnitude $\beta^{\log}(x)$ and set

$$RR_{tC}(x)=\left(\frac{P_t(x)}{P_C(x)}\right)^{-\beta^{\log}(x)}.$$

Equivalently, with

$$z_t(x)=\log\frac{P_t(x)}{P_C(x)},$$

we have

$$\log RR_{tC}(x)=-\beta^{\log}(x)z_t(x).$$

This is the default scalar model in the tutorial. It says that point elasticity is approximately constant over the tested range.

### Constant standard-relative-elasticity model

If the organization wants the standard finite elasticity itself to be the shared model parameter, define

$$\delta_t(x)=\frac{P_t(x)}{P_C(x)}-1$$

and predict a nonnegative magnitude $\beta^{rel}(x)$ such that

$$RR_{tC}(x)=1-\beta^{rel}(x)\delta_t(x).$$

For nominal $\pm 10\,\%$ doses,

$$RR_{HC}(x)=1-0.10\,\beta^{rel}(x),$$
$$RR_{LC}(x)=1+0.10\,\beta^{rel}(x).$$

This is a **linear relative-demand model**, not a constant point-elasticity model. Positivity requires

$$1-\beta^{rel}(x)\delta_t(x)>0$$

for every tested arm. With a fixed $+10\,\%$ high arm, this requires

$$0\le\beta^{rel}(x)<10.$$

A safe raw-score parameterization is

$$\beta^{rel}(x)=B\,\sigma\big(f(x)\big),$$

where $B$ is chosen slightly below the smallest positivity bound implied by any high-price dose in the modeling population.

The corresponding buyer posterior is

$$r_t(x)=\frac{\pi_t(x)\big[1-\beta^{rel}(x)\delta_t(x)\big)}{\sum_j \pi_j(x)\big[1-\beta^{rel}(x)\delta_j(x)\big)}.$$

Its natural training loss is still buyer categorical cross-entropy:

$$\ell_i=-\log r_{T_i}(X_i).$$

The log-constant and relative-linear scalar models imply different shapes between and beyond the tested points. Compare them by strictly out-of-fold buyer likelihood, randomized grouped calibration, stability, and policy value. Do not select one merely because its parameter uses the organization's preferred reporting units.

## 5.12 Buyer posterior under the log-constant scalar model

Under nominal multipliers, define

$$z_H = \log 1.10 , z_C = 0 , z_L = \log 0.90 .$$

Insert the scalar risk ratios into the Bayes-flip identity:

$$r_t(x) = \frac{\pi_t(x) \exp\{-\beta^{\log}(x) z_t(x)\}}{\sum_j \pi_j(x) \exp\{-\beta^{\log}(x) z_j(x)\}} .$$

Equivalently,

$$r(x) = softmax\left(\log \pi(x) - \beta^{\log}(x) z(x)\right).$$

Here $\log \pi(x)$ and $z(x)$ are three-element vectors, and the logarithm is applied componentwise.

This parameterization guarantees positive probabilities that sum to one and monotone demand whenever $\beta^{\log}(x) \geq 0$ and prices are ordered correctly.

## 5.13 Two-gap model

Either scalar model can be too restrictive if the response to a $+10\%$ increase differs from the response to a $-10\%$ decrease. A two-gap model predicts

$$\ell_H(x) = \log RR_{HC}(x), \ell_L(x) = \log RR_{LC}(x),$$

with

$$\ell_H(x) \leq 0 , \ell_L(x) \geq 0 .$$

The buyer posterior is

$$r(x) = softmax\left(\log \pi(x) + \left[\ell_H(x), 0, \ell_L(x)\right]\right).$$

After fitting, derive either elasticity convention:

$$\varepsilon^{\log}_{HC}(x) = \frac{\ell_H(x)}{\log\left(P_H/P_C\right)}, \varepsilon^{\log}_{LC}(x) = \frac{\ell_L(x)}{\log\left(P_L/P_C\right)},$$

and

$$\varepsilon^{rel}_{HC}(x) = \frac{e^{\ell_H(x)} - 1}{P_H/P_C - 1}, \varepsilon^{rel}_{LC}(x) = \frac{e^{\ell_L(x)} - 1}{P_L/P_C - 1} .$$

The two-gap model is more flexible but has higher variance, especially because the side-arm buyer counts are small. It should usually be regularized toward one of the scalar restrictions rather than treated as automatically superior.

## 5.14 Asymmetry is not automatically individual curvature

Suppose different latent subpopulations have different constant elasticities. Averaging their conversion probabilities can make the **marginal** high-side and low-side finite differences unequal, even if each latent customer follows a constant-elasticity curve.

In addition, the standard base-period definition itself produces different high- and low-side values under a perfectly constant point-elasticity curve, as Section 5.6 showed.

Therefore, finding different high- and low-side elasticities does not by itself prove that an individual demand curve is curved or asymmetric. It can reflect:

- the chosen finite-change convention;
- unobserved heterogeneity;
- different customer mixtures among buyers at each arm;
- rounding or noncompliance;
- time or channel imbalance;
- estimation noise.

Interpret two-gap asymmetry as a marginal finite-difference feature of the experiment unless stronger structural assumptions are justified.

### 5.15 Nominal versus realized price dose

If the experiment is a clean multiplier test, use the randomized nominal doses for the primary intention-to-treat scalar model. If the planned counterfactual prices can be computed deterministically from pre-treatment information and the experimental rule, row-specific doses may be used:

$$z_{i,t} = \log \frac{P_{i,t}}{P_{i,C}},$$

for the log-constant model, or

$$\delta_{i,t} = \frac{P_{i,t}}{P_{i,C}} - 1,$$

for the standard relative-change model and reporting transformation.

Do not use a post-assignment realized price variable that is affected by manual intervention or customer behavior without explicitly analyzing compliance.

## 6. Business targeting scores and price decisions

### 6.1 Relative contribution factor

Define

$$B_t(x) = \frac{g_t(x)}{g_C(x)} R R_{tC}(x),$$

provided $g_C(x) > 0$. $B_t(x)$ is the expected contribution under arm $t$ relative to control, after the common conversion scale has canceled.

Interpretation:

- $B_t(x) > 1$: arm $t$ is predicted to improve contribution relative to control;
- $B_t(x) = 1$: predicted tie;
- $B_t(x) < 1$: control is predicted to be better.

The relative lift is

$$B_t(x) - 1.$$

### 6.2 Log targeting score

A numerically stable equivalent is

$$S_t(x) = \log \frac{g_t(x)}{g_C(x)} + \log R R_{tC}(x).$$

Because logarithm is monotone,

$$S_t(x) > 0 \Leftrightarrow B_t(x) > 1.$$

The high-price score is

$$S_H(x) = \log \frac{g_H(x)}{g_C(x)} + \ell_H(x),$$

and the low-price score is

$$S_L(x) = \log \frac{g_L(x)}{g_C(x)} + \ell_L(x).$$

These scores are often more operationally useful than elasticity alone. An elasticity of three may be acceptable for a large margin increase and unacceptable for a small one.

## 6.3 Decision rule with three arms

Set

$$RR_{CC}(x)=1.$$

Calculate the three scores

$$A_H(x)=g_H(x)RR_{HC}(x),$$
$$A_C(x)=g_C(x),$$
$$A_L(x)=g_L(x)RR_{LC}(x).$$

Then select

$$d^{\dot{c}}(x)=arg\,max\{A_H(x),A_C(x),A_L(x)\}.$$

## 6.4 Uncertainty-aware action

A point prediction is not enough when price decisions are costly or regulated. A conservative policy can require the lower confidence bound of relative value to be positive.

For the high arm, for example, act only when

$$LCB_{1-\alpha}\{S_H(x)\}>0,$$

where $LCB$ denotes a lower confidence bound and $\alpha$ is a chosen error level.

Individual confidence intervals are difficult for complex machine-learning models. Practical alternatives include:

- fold and seed stability;
- bootstrap distributions of segment-level scores;
- conformal or ensemble uncertainty methods;
- an abstention band around zero;
- limiting personalized actions to large, stable groups.

## 6.5 Portfolio constraints

Independent quote-level maximization may violate business constraints. Examples include:

- only a fixed percentage of quotes may receive a price increase;
- a minimum expected sales volume must be retained;
- price changes must remain within legal or governance limits;
- protected or sensitive variables may not be used;
- a channel or tariff segment may have an exposure cap.

Then the policy becomes a constrained optimization problem. A common procedure is:

1. calculate $S_H(x)$ and $S_L(x)$ for every eligible quote;
2. rank quotes by conservative expected gain;
3. allocate actions subject to the portfolio constraints;
4. evaluate the complete constrained rule using randomized policy-value estimators.

## 6.6 Selection and claim-cost caveat

The contribution $g_t(x)$ must be arm-specific if price changes the risk mix of purchasers. A higher price may cause safer or riskier customers to leave disproportionately. If expected claim cost conditional on purchase depends on arm beyond the observed $x$, then using a fixed technical cost can misstate contribution.

This issue does not invalidate elasticity estimation, but it changes the business optimization layer. Claim-cost selection should be analyzed with appropriate delayed-outcome methods and governance; it must not be inferred from the short-term conversion model alone.

# 7. Real-data contract and experiment audit

## 7.1 Required row structure

The analysis should begin with one row per eligible randomized quote, including buyers and nonbuyers. A minimal schema contains:

| Field | Meaning |
|---|---|
| `quote_id` | Unique experimental quote opportunity |
| `customer_id` or `household_id` | Stable grouping identifier for repeated quotes |
| `quote_timestamp` | Time of randomization or price display |
| `eligible` | Whether the quote belonged to the pre-defined experiment population |
| `arm` | Randomized arm: H, C, or L |
| `purchase` | Binary outcome within the fixed window |
| `pi_H`, `pi_C`, `pi_L` | Row-level assignment propensities |
| `base_premium` | Premium before the experimental perturbation |
| `assigned_multiplier` | Intended multiplier or randomized dose |
| `displayed_premium` | Price actually shown, for compliance audit |
| `tariff_version` | Version or block relevant to randomization and drift |
| `contribution_H/C/L` | Contribution if bought under each tested action |
| pre-treatment features | Explicitly approved customer, vehicle, channel, tariff, and market variables |

## 7.2 Immediate stop conditions

Do not fit elasticity models when any of the following is unresolved:

- propensities are absent, nonpositive, or fail to sum to one;
- an arm has no eligible quotes or no buyers;
- assignment was not randomized, or the randomizer cannot be reconstructed;
- rows were removed after observing arm or outcome;
- the outcome window differs by arm or calendar time;
- feature timing is unknown;
- repeated customers exist but no grouping identifier is available;
- the implementation changed without a logged block, date, or version;
- displayed price did not follow assignment and no compliance analysis is possible.

## 7.3 Population accounting

Create a flow table that reports:

1. records received;
2. records eligible before randomization;
3. randomized quotes by arm;
4. outcomes observed within the window;
5. buyers and nonbuyers by arm;
6. unique customers and repeated quote episodes;
7. every exclusion, its timing, and its reason.

Heterogeneity capacity should be judged using **side-arm buyer counts**, not total quote count. A dataset with 100,000 quotes and $5\%$ conversion may contain only a few hundred high-arm buyers.

## 7.4 Assignment audit

For arm $t$, compare the observed assignment count

$$O_t = \sum_i 1\left(T_i = t\right)$$

with the expected count

$$E_t = \sum_i \pi_{i,t}.$$

Here $1(\cdot)$ is the indicator function: it equals one when its condition is true and zero otherwise.

A standardized residual is

$$Z_t = \frac{O_t - E_t}{\sqrt{\sum_i \pi_{i,t}(1-\pi_{i,t})}}.$$

Inspect residuals by week, block, tariff version, channel, geography, device, and premium band. Systematic patterns suggest an implementation or logging problem, not useful heterogeneity.

## 7.5 Covariate balance

Randomization should make pre-treatment covariates similar across arms in expectation. For a numeric feature $X_k$, a simple standardized mean difference between arm $t$ and control is

$$SMD_{tC,k} = \frac{\acute{X}_{t,k} - \acute{X}_{C,k}}{s_k},$$

where $s_k$ is a pooled or overall standard deviation.

Some imbalance occurs by chance, especially with many variables. The purpose is not to run hundreds of significance tests but to detect large, systematic, or time-structured deviations from the randomization design.

## 7.6 Leakage audit

A feature is **leakage** if it contains information created after assignment or after price display. Reject:

- arm, assigned multiplier, or displayed experimental price as a feature in the buyer response model;
- a feature calculated using the displayed price after randomization;
- payment choice, policy identifier, purchase channel behavior, cancellation, or claims observed after purchase;
- agent or customer actions that occurred after seeing the price;
- target encodings built from buyer arm labels outside a fold-local procedure.

Potentially valid pre-treatment features include the unmodified base premium, technical risk cost, tariff uncertainty, competitor information observed before assignment, channel, acquisition source, renewal status, and market-time variables.

## 7.7 Intent-to-treat compliance report

For each arm, compare

$$\frac{\text{displayed premium}}{\text{base premium}}$$

with the assigned multiplier. Report mean and tail absolute errors, failed displays, overrides, and minimum-premium effects.

The primary model should retain the randomized arm even when compliance is imperfect. A secondary dose-response model can be added only with an explicit causal strategy.

# 8. Validation architecture

## 8.1 Why ordinary random train/test splits are inadequate

A conventional random row split can leak information when one customer appears several times. It can also hide time drift and allow hyperparameter tuning to adapt to the final test set.

The validation design must protect against:

- customer or household leakage;

- time leakage;
- preprocessing leakage;
- calibration leakage;
- hyperparameter-selection bias;
- repeated inspection of the final holdout.

## 8.2 Final chronological holdout

Reserve the most recent $15\,\%$ to $25\,\%$ of customer or household clusters as a final holdout. Assign a cluster to a period using a pre-specified timestamp rule, such as the latest eligible quote time.

The final holdout is used once, after model families, search spaces, metrics, and selection rules have been frozen.

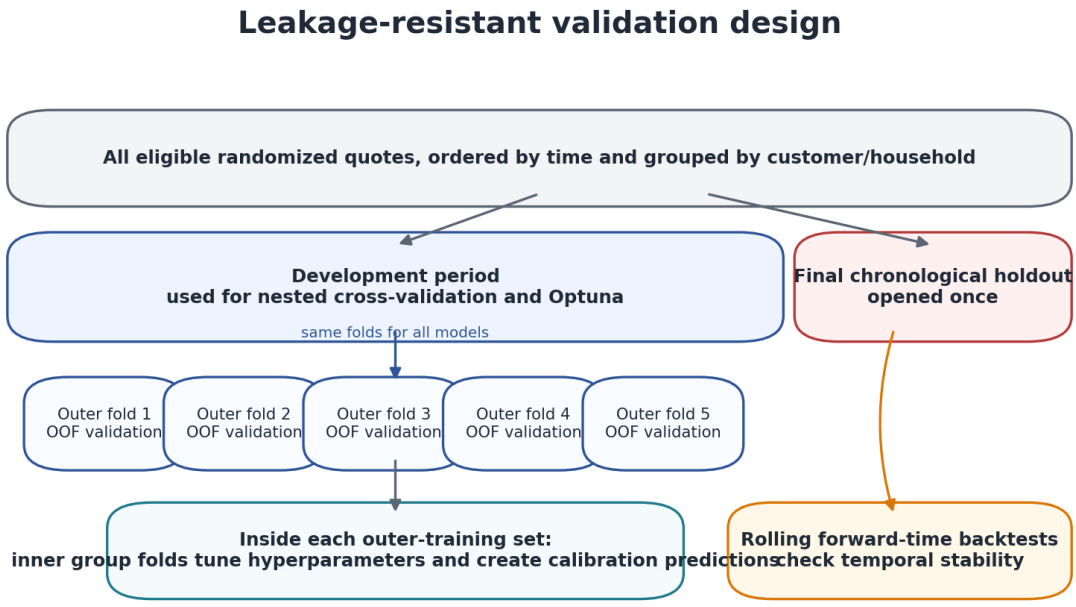
## 8.3 Nested cross-validation

Within the development period, use two levels of cross-validation:

- **outer folds** estimate out-of-sample model quality;
- **inner folds** select hyperparameters and generate calibration predictions.

A recommended design is five outer group-disjoint folds and four inner group-disjoint folds, adjusted when the number of side-arm buyers is insufficient.

`StratifiedGroupKFold` attempts to preserve arm proportions while ensuring non-overlapping groups [5]. Stratification is an engineering aid, not a guarantee that every small fold contains adequate side-arm buyers. Fold counts must be checked explicitly.




*Nested, group-disjoint, and time-aware validation*

## 8.4 Outer-fold workflow

For each outer fold:

1. hold out the outer-validation customers;
2. tune hyperparameters using only the outer-training customers and inner folds;
3. create inner out-of-fold raw predictions on the outer-training set;
4. fit the calibrator on those inner out-of-fold predictions;
5. refit the base model on all outer-training buyers;
6. predict the outer-validation buyers;
7. apply the already fitted calibrator;
8. store raw and calibrated outer out-of-fold predictions.

At no point may a calibrator see predictions from observations used to fit the corresponding base model.

## 8.5 Forward-time backtests

In addition to grouped cross-validation, run rolling forward-time backtests. Train on an earlier period and validate on the next period, repeating at several cutoffs.

A model that improves random grouped cross-validation but loses its effect ranking in forward time is not production eligible. Insurance tariffs, competitor prices, channels, and customer mix can drift.

## 8.6 Identical folds for all candidates

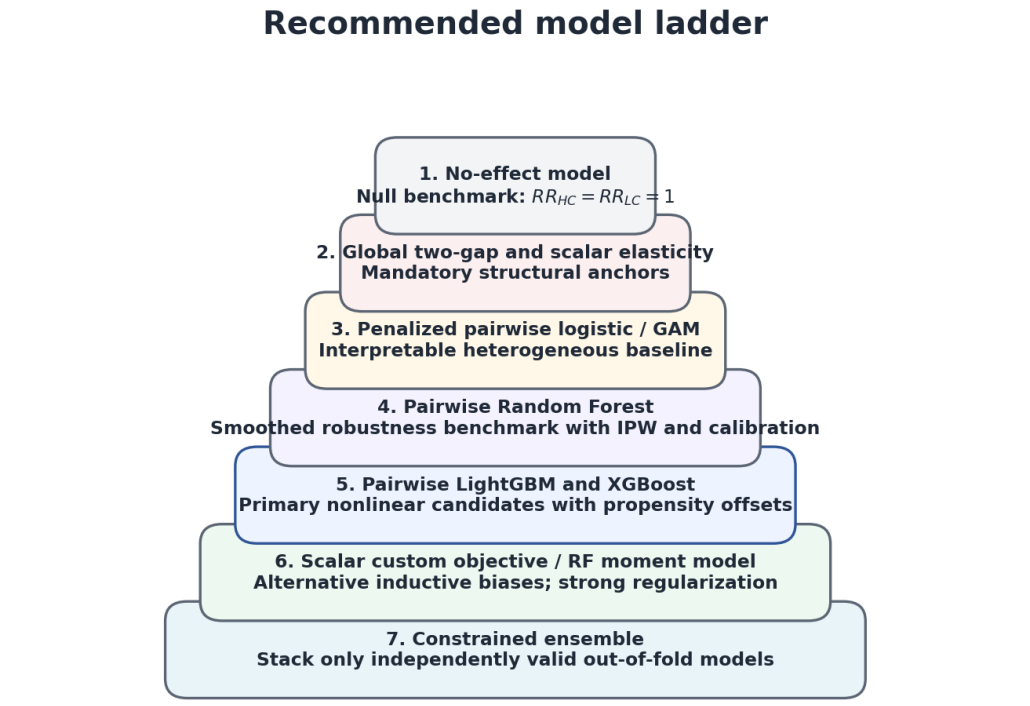
Every model must use the same outer folds. Paired comparisons are much more precise than comparisons across unrelated splits.

Store the fold assignment as an artifact and reuse it for:

- all model families;
- both H/C and L/C contrasts;
- calibration;
- ensembling;
- policy evaluation;
- stability analysis.

# 9. Recommended model ladder

A disciplined project fits models from simplest to most flexible. The purpose of the ladder is not bureaucracy; it makes the evidence for heterogeneity explicit.




*A complex model is eligible only after it beats the simpler anchor out of fold, remains calibrated, and improves randomized business value.*

*Recommended model ladder*

| Level | Model | Main purpose |
|---|---|---|
| 1 | No-effect model | Null benchmark and experiment sanity check |
| 2 | Global two-gap model | Portfolio H/C and L/C risk ratios without heterogeneity |
| 2 | Global scalar elasticity | Strong structural anchor using all three buyer arms |

| Level | Model | Main purpose |
|---|---|---|
| 3 | Penalized pairwise logistic model | Interpretable heterogeneous baseline |
| 4 | Pairwise Random Forest | Smoothed, algorithmically different robustness benchmark |
| 5 | Pairwise LightGBM | Primary nonlinear candidate with exact propensity offset |
| 5 | Pairwise XGBoost | Primary nonlinear candidate with exact propensity offset |
| 6 | Scalar custom-objective boosting | Nonlinear model that preserves one-dimensional elasticity structure |
| 6 | Random-Forest moment inversion | Alternative scalar benchmark without invented elasticity labels |
| 7 | Constrained ensemble | Variance control using independently valid out-of-fold predictions |

A higher level is not automatically better. Personalized elasticity is accepted only when it provides reproducible out-of-fold improvement over the global anchor.

# 10. Global baseline models

## 10.1 No-effect model

The null hypothesis of no price response is

$$q_H(x)=q_C(x)=q_L(x).$$

Then

$$RR_{HC}(x)=RR_{LC}(x)=1,$$

and the buyer posterior equals the assignment propensity:

$$r_t(x)=\pi_t(x).$$

This is the correct null predictor for buyer cross-entropy. It is more informative than a uniform one-third predictor because the experiment did not assign arms uniformly.

## 10.2 Global two-gap model

The global two-gap model assumes constant portfolio-level log risk ratios:

$$\ell_H(x)=\ell_H,\ell_L(x)=\ell_L.$$

The pairwise buyer likelihoods estimate $\ell_H$ and $\ell_L$ separately. Monotonicity can be imposed as

$$\ell_H\le 0,\ell_L\ge 0.$$

This model answers:

- how much does the high arm change conversion on average relative to control?
- how much does the low arm change conversion on average relative to control?
- is the upper response consistent with the lower response?

The model is an essential reference even if the final system is personalized.

## 10.3 Global scalar elasticity

The global scalar model uses one constant $\beta\ge 0$:

$$r_{i,t}(\beta)=\frac{\pi_{i,t}e^{-\beta z_{i,t}}}{\sum_j \pi_{i,j}e^{-\beta z_{i,j}}}.$$

For buyer $i$, the observed arm is $T_i$. Its negative log-likelihood contribution is

$$\ell_i(\beta)=-\log r_{i,T_i}(\beta).$$

Expanding the expression gives

$$\ell_i(\beta)=\beta z_{i,T_i}+\log\sum_j \pi_{i,j}e^{-\beta z_{i,j}}-\log \pi_{i,T_i}.$$

The last term is known and does not affect the optimizing value of $\beta$.

The derivative is

$$\frac{\partial \ell_i}{\partial \beta}=z_{i,T_i}-E_{r_i(\beta)}[z_i],$$

where

$$E_{r_i(\beta)}[z_i]=\sum_j r_{i,j}(\beta)z_{i,j}.$$

The second derivative is

$$\frac{\partial^2 \ell_i}{\partial \beta^2}=Var_{r_i(\beta)}(z_i)\geq 0.$$

Therefore, the buyer loss is convex in the scalar $\beta$. A one-dimensional bounded optimizer is sufficient for the global model.

## 10.4 Python: global scalar fit

The reference package provides `fit_global_scalar_beta`. The essential pattern is:

```
import numpy as np
from elasticity_only.scalar import fit_global_scalar_beta

# Use buyers only for the flip likelihood.
buyer = df["purchase"].eq(1).to_numpy()

arm_buyer = df.loc[buyer, "arm"].to_numpy()
pi_buyer = df.loc[buyer, ["pi_H", "pi_C", "pi_L"]].to_numpy(float)

# Nominal intention-to-treat log price doses.
z = np.log(np.array([1.10, 1.00, 0.90], dtype=float))

fit = fit_global_scalar_beta(
    arm=arm_buyer,
    pi=pi_buyer,
    z=z,
    beta_upper=30.0,
)

print(f"beta = {fit.beta:.4f}")
print(f"mean buyer NLL = {fit.nll:.6f}")
print(f"converged = {fit.converged}")
```

`beta_upper` is a numerical search boundary, not a claim that true elasticity cannot exceed that value. Sensitivity to the boundary should be checked when the optimum is near it.

## 10.5 Uncertainty for global models

Suitable uncertainty methods include:

- an observed-information or sandwich standard error;
- a profile-likelihood interval;

- a customer- or household-cluster bootstrap;
- a Bayesian posterior with a weakly informative nonnegative prior.

Because repeat quotes violate row independence, a cluster bootstrap is often the safest general-purpose method. Resample customers or households, not individual rows.

### 10.6 Global-anchor decision rule

The global scalar model produces the same $\beta$ for every quote but different business decisions when $g_t(x)$ differs. It is therefore not necessarily a uniform-price policy.

For each quote:

1. calculate global $RR_{HC}=e^{-\beta z_H}$ and $RR_{LC}=e^{-\beta z_L}$;
2. combine them with quote-specific contribution $g_t(x)$;
3. choose the arm with maximum $g_t(x)RR_{tC}$.

A flexible elasticity model must improve over this economically meaningful anchor, not merely over the no-effect model.

## 11. Pairwise buyer models

### 11.1 Why pairwise modeling is exact

Fix a side arm $t \in \{H, L\}$. Restrict attention to buyers who were assigned either arm $t$ or control. Define the binary target

$$Z_{i,t}=\begin{cases}1, & T_i=t,\\ 0, & T_i=C.\end{cases}$$

Among these pairwise buyers,

$$P(Z_{i,t}=1 \mid Y_i=1, T_i \in \{t, C\}, X_i=x)=\frac{\pi_{i,t}q_t(x)}{\pi_{i,t}q_t(x)+\pi_{i,C}q_C(x)}.$$

Take the odds:

$$\frac{P(Z_{i,t}=1 \mid \cdots)}{P(Z_{i,t}=0 \mid \cdots)}=\frac{\pi_{i,t}q_t(x)}{\pi_{i,C}q_C(x)}.$$

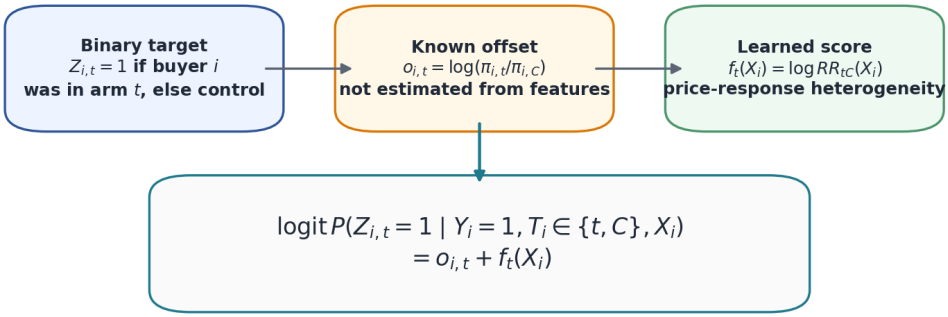
Take logarithms:

$$\operatorname{logit} P(Z_{i,t}=1 \mid \cdots)=\log\frac{\pi_{i,t}}{\pi_{i,C}}+\log RR_{tC}(x).$$

This is an ordinary binary logistic likelihood with:

- a known offset $\log(\pi_{i,t}/\pi_{i,C})$;
- a learned score $f_t(x)=\log RR_{tC}(x)$.

**Pairwise buyer model: known randomization odds plus learned log risk ratio**




For XGBoost use base_margin; for LightGBM use init_score. For generic scikit-learn classifiers, inverse-propensity weighting is an alternative when a fixed offset is unavailable.

*Exact pairwise offset decomposition*

## 11.2 Why the two pairwise models are coherent

Fit one model for H/C and one for L/C. Let their outputs be

$$\ell_H(x) = \log RR_{HC}(x), \ell_L(x) = \log RR_{LC}(x).$$

Reconstruct the three-arm buyer posterior as

$$r(x) = softmax\left(\log \pi(x) + \left[\ell_H(x), 0, \ell_L(x)\right]\right).$$

This reconstruction guarantees that $r_H + r_C + r_L = 1$.

Two **one-versus-rest** classifiers would not have this property. Separate one-versus-rest probabilities can be mutually inconsistent and need arbitrary renormalization. The pairwise-with-control formulation estimates the two desired risk ratios directly and then reconstructs one coherent posterior.

## 11.3 Pairwise negative log-likelihood

For a pairwise buyer with binary label $Z_i$ and linear predictor

$$\eta_i = o_i + f(X_i),$$

where

$$o_i = \log(\pi_{i,t}/\pi_{i,C}),$$

its negative log-likelihood is

$$L_i = \log(1 + e^{\eta_i}) - Z_i \eta_i.$$

The numerically stable Python expression is

```
np.logaddexp(0.0, eta) - y * eta
```

## 11.4 Equal-contrast objective

The H/C and L/C tasks can have different sample sizes and difficulty. For joint comparison, use

$$NLL_{\text{equal}} = \frac{1}{2} NLL_{HC} + \frac{1}{2} NLL_{LC}.$$

This prevents the easier or larger contrast from dominating model selection.

## 11.5 Exact offset versus inverse-propensity weighting

### Offset formulation

The offset formulation fits the observed pairwise buyer data without changing their distribution:

$$\operatorname{logit} p_i = o_i + f(X_i).$$

It is statistically natural and should be preferred when the library supports row-level offsets.

### Inverse-propensity formulation

Suppose pairwise buyers are weighted by

$$w_i = \frac{1}{\pi_{i,T_i}}.$$

In the weighted population,

$$P_w(T = t \mid Y = 1, T \in \{t, C\}, X = x) = \frac{q_t(x)}{q_t(x) + q_C(x)}.$$

Therefore,

$$\operatorname{logit} P_w(T = t \mid \cdots) = \log RR_{tC}(x).$$

A standard binary classifier trained with these weights can estimate the log risk ratio without an explicit offset.

The weighting approach is useful for scikit-learn estimators and Random Forest, but it has costs:

- side-arm observations receive much larger weights under 10/80/10 assignment;
- the effective sample size can decline;
- tree splits can become sensitive to a small number of high-weight buyers;
- ordinary probability-calibration tools may not preserve the desired weighted estimand unless used carefully.

## 11.6 Do not add class balancing

Class imbalance among buyers is partly the causal signal. Using `class_weight="balanced"`, `scale_pos_weight`, naive oversampling, or one-third class balancing changes the target posterior.

If sampling or weighting is intentionally used for computational reasons, it must be reversed analytically. Suppose class $t$ is retained with probability $s_t$. The sampled posterior is

$$\tilde{r}_t(x) = \frac{s_t r_t(x)}{\sum_j s_j r_j(x)}.$$

Recover the original posterior using

$$r_t(x) = \frac{\tilde{r}_t(x)/s_t}{\sum_j \tilde{r}_j(x)/s_j}.$$

Duplicating buyers or applying SMOTE does not create experimental information.

# 12. Penalized pairwise logistic regression

## 12.1 Why it is an important baseline

Penalized logistic regression is often the best first heterogeneous model because it is:

- interpretable;
- stable under small side-arm counts;
- easy to regularize;
- fast enough for nested cross-validation;
- a strong diagnostic for whether the experiment supports any reproducible heterogeneity.

The model can include:

- standardized numeric main effects;
- one-hot encoded categorical variables;

- a small, pre-specified set of interactions;
- spline or binned transformations for important continuous variables.

## 12.2 Linear predictor

For contrast $t/C$,

$$f_t(x) = \alpha_t + x^\top \theta_t.$$

Here:

- $\alpha_t$ is an intercept representing the average log risk ratio;
- $x$ is the feature vector;
- $\theta_t$ is the coefficient vector;
- $x^\top \theta_t$ denotes the dot product.

The full pairwise logit is

$$\eta_{i,t} = \log \frac{\pi_{i,t}}{\pi_{i,C}} + \alpha_t + x_i^\top \theta_t.$$

## 12.3 Regularization

An L2 penalty is

$$\lambda \sum_k \theta_k^2.$$

It shrinks coefficients smoothly toward zero.

An L1 penalty is

$$\lambda \sum_k |\theta_k|.$$

It can set coefficients exactly to zero.

Elastic net combines the two:

$$\lambda \left[ (1-\rho) \frac{1}{2} \sum_k \theta_k^2 + \rho \sum_k |\theta_k| \right],$$

where $\rho \in [0, 1)$ is the L1 ratio.

In scikit-learn, the parameter `C` is approximately the inverse of regularization strength: smaller `C` means stronger regularization.

## 12.4 Offset implementation options

`sklearn.linear_model.LogisticRegression` does not expose a general row-level fixed offset. Three practical options are:

1. use the inverse-propensity weighted target and fit ordinary logistic regression;
2. implement the offset likelihood with `scipy.optimize.minimize` and wrap it as a scikit-learn estimator;
3. use a generalized-linear-model library that supports offsets, while keeping scikit-learn preprocessing and validation.

For a portable scikit-learn-only baseline, the IPW formulation is acceptable if weights and calibration are audited.

## 12.5 Fold-local preprocessing

A safe preprocessing pipeline for linear models is:

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
```

```
    ("scale", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        handle_unknown="infrequent_if_exist",
        min_frequency=20,
    )),
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipe, numeric_features),
    ("categorical", categorical_pipe, categorical_features),
])
```

Every preprocessing object must be fitted inside the training fold. Rare-level thresholds can be tuned, but must never be learned from the outer validation or final holdout.

### 12.6 Optuna search for logistic regression

A practical elastic-net search is:

```
def suggest_logistic(trial):
    return {
        "C": trial.suggest_float("C", 1e-4, 100.0, log=True),
        "l1_ratio": trial.suggest_float("l1_ratio", 0.0, 1.0),
        "solver": "saga",
        "max_iter": 5000,
    }
```

Also consider tuning:

- the minimum frequency for rare categorical levels;
- whether a small approved interaction set is included;
- spline degrees of freedom for a few continuous features;
- a maximum feature count selected within inner folds.

Do not search thousands of arbitrary interactions. With weak treatment signal, the search process itself can overfit.

## 13. Pairwise LightGBM models

### 13.1 What LightGBM is doing

LightGBM is a gradient-boosted decision-tree library. A **decision tree** partitions the feature space into regions and assigns a constant score to each terminal region, called a **leaf**. A **boosted ensemble** builds many trees sequentially. Each new tree is chosen to reduce the remaining loss of the current ensemble.

For the H/C contrast, the desired model is

$$\mathit{logit}\, p_{i,H} = \log \frac{\pi_{i,H}}{\pi_{i,C}} + f_H(X_i),$$

where

$$p_{i,H} = P(T_i = H \mid Y_i = 1, T_i \in \{H, C\}, X_i).$$

The function $f_H(x)$ is interpreted as

$$f_H(x) = \log RR_{HC}(x).$$

The L/C model is identical after replacing $H$ by $L$.

LightGBM calls a per-row initial raw score `init_score`. In this application the initial score is not a loose initialization that can be discarded. It represents a known offset and a global price-response anchor. The trees should learn only the residual heterogeneous response.

## 13.2 Why add a global anchor to the offset

The pure propensity offset is

$$o_i = \log\left(\pi_{i,t}/\pi_{i,C}\right).$$

Let $\hat{\ell}_t$ be the global pairwise log risk ratio obtained from the training fold. Fit the tree residual around

$$o_i + \hat{\ell}_t.$$

The complete fitted log risk ratio becomes

$$\widehat{\log RR}_{tC}(x) = \hat{\ell}_t + F_t(x),$$

where $F_t(x)$ is the sum of the tree contributions. This decomposition is beneficial because:

- the intercept-like portfolio effect is estimated by a low-variance global model;
- the trees only need to explain deviations from that effect;
- early stopping can leave the model close to the global anchor when heterogeneity is weak;
- the raw tree score has a clean interpretation as a heterogeneous residual.

The global anchor must be estimated inside the current training fold. Computing it once on all data would leak information into validation.

## 13.3 Native LightGBM implementation pattern

```
from __future__ import annotations

import numpy as np
import lightgbm as lgb
from scipy.optimize import minimize_scalar



def fit_global_pairwise_log_rr(
    pair_arm: np.ndarray,
    pair_y: np.ndarray,
    pi_t: np.ndarray,
    pi_c: np.ndarray,
    sample_weight: np.ndarray | None = None,
) -> float:
    """Fit one constant log risk ratio using the exact offset likelihood."""
    offset = np.log(pi_t) - np.log(pi_c)
    w = np.ones(len(pair_y)) if sample_weight is None else sample_weight

    def objective(log_rr: float) -> float:
        eta = offset + log_rr
        rows = np.logaddexp(0.0, eta) - pair_y * eta
        return float(np.average(rows, weights=w))

    result = minimize_scalar(
        objective,
        bounds=(-12.0, 12.0),
        method="bounded",
    )
    if not result.success:
        raise RuntimeError("Global pairwise optimization failed")
    return float(result.x)



def fit_lgb_pairwise_offset(
    X_train,
    arm_train: np.ndarray,
    pi_train: np.ndarray,
    X_valid,
    arm_valid: np.ndarray,
    pi_valid: np.ndarray,
    *,
    contrast_arm: int,
    params: dict,
```

```
    num_boost_round: int = 5000,
    early_stopping_rounds: int = 150,
):
    """
    Fit H/C when contrast_arm=0 or L/C when contrast_arm=2.
    Arm coding is [H=0, C=1, L=2].
    """
    if contrast_arm not in (0, 2):
        raise ValueError("contrast_arm must be 0 for H or 2 for L")

    train_mask = np.isin(arm_train, [contrast_arm, 1])
    valid_mask = np.isin(arm_valid, [contrast_arm, 1])

    y_train = (arm_train[train_mask] == contrast_arm).astype(float)
    y_valid = (arm_valid[valid_mask] == contrast_arm).astype(float)

    off_train = (
        np.log(pi_train[train_mask, contrast_arm])
        - np.log(pi_train[train_mask, 1])
    )
    off_valid = (
        np.log(pi_valid[valid_mask, contrast_arm])
        - np.log(pi_valid[valid_mask, 1])
    )

    global_log_rr = fit_global_pairwise_log_rr(
        arm_train[train_mask],
        y_train,
        pi_train[train_mask, contrast_arm],
        pi_train[train_mask, 1],
    )

    train_set = lgb.Dataset(
        X_train.iloc[np.flatnonzero(train_mask)],
        label=y_train,
        init_score=off_train + global_log_rr,
        free_raw_data=False,
    )
    valid_set = lgb.Dataset(
        X_valid.iloc[np.flatnonzero(valid_mask)],
        label=y_valid,
        init_score=off_valid + global_log_rr,
        reference=train_set,
        free_raw_data=False,
    )

    complete_params = {
        "objective": "binary",
        "metric": "binary_logloss",
        "boosting_type": "gbdt",
        "learning_rate": 0.03,
        "num_leaves": 15,
        "max_depth": 5,
        "min_data_in_leaf": 150,
        "min_sum_hessian_in_leaf": 5.0,
        "feature_fraction": 0.8,
        "bagging_fraction": 0.8,
        "bagging_freq": 1,
        "lambda_l1": 0.0,
        "lambda_l2": 10.0,
        "min_gain_to_split": 0.0,
        "boost_from_average": False,
        "feature_pre_filter": False,
        "verbosity": -1,
        "seed": 2026,
        "num_threads": 1,
        **params,
    }
```

```
    booster = lgb.train(
        complete_params,
        train_set,
        num_boost_round=num_boost_round,
        valid_sets=[valid_set],
        valid_names=["valid"],
        callbacks=[
            lgb.early_stopping(early_stopping_rounds, verbose=False),
        ],
    )
    return booster, global_log_rr


def predict_lgb_log_rr(booster, global_log_rr: float, X) -> np.ndarray:
    """Return log RR, not pairwise buyer probability."""
    residual = booster.predict(
        X,
        raw_score=True,
        num_iteration=booster.best_iteration,
    )
    return global_log_rr + np.asarray(residual, dtype=float)
```

The important prediction distinction is:

- `raw_score=True` returns the tree margin;
- adding the global anchor gives $\widehat{\log RR}$;
- adding the row-level propensity offset and applying the logistic function gives the pairwise buyer probability.

For a new row,

$$\hat{p}_{tC}(x)=\sigma\left[\log\frac{\pi_t(x)}{\pi_C(x)}+\widehat{\log RR}_{tC}(x)\right].$$

Do not accidentally evaluate buyer log loss on $\sigma\left(\widehat{\log RR}\right)$ when the experimental propensities are unequal. The known offset must be included in the probability used for likelihood evaluation.

## 13.4 Important LightGBM hyperparameters

**`learning_rate`**
The learning rate multiplies the contribution of every new tree. Smaller values usually require more trees but make the optimization path smoother. For weak treatment-effect signal, values between approximately $0.005$ and $0.10$ are sensible.

**`num_leaves`**
`num_leaves` limits the number of terminal regions in one tree. It is a direct complexity control. A tree with many leaves can isolate small groups of side-arm buyers and produce extreme, unstable log risk ratios.

**`max_depth`**
`max_depth` limits the number of split levels. For this problem, depths from 2 to 7 are usually broad enough for tuning. Very deep trees are hard to justify unless the buyer sample is exceptionally large.

**`min_data_in_leaf`**
This is one of the most important parameters. It controls the minimum number of training observations in a leaf. The bound should depend on the number of pairwise buyers and especially on the minority side-arm proportion.

A useful dynamic lower bound is

$$m_{min}=max\left(20,\lceil\frac{10}{\rho_{min}}\rceil\right),$$

where $\rho_{min}$ is the minority-arm fraction among the pairwise buyers. The heuristic aims for roughly ten minority-arm buyers in a typical minimally sized leaf. It is not a theorem; it is a conservative starting point that Optuna may tune upward.

**`min_sum_hessian_in_leaf`**
The Hessian is the local curvature of the loss. This parameter prevents splits whose child leaves contain too little curvature, and therefore too little effective information. Search it on a logarithmic scale, for example $0.1$ to $50$.

**`feature_fraction`**
This is the fraction of features considered by each tree. It can reduce correlation between trees and protect against the same noisy feature winning every split. A range of $0.5$ to $1.0$ is suitable.

**`bagging_fraction` and `bagging_freq`**
`bagging_fraction` specifies the fraction of rows used in a bagging iteration. `bagging_freq` controls how often a new row sample is drawn. A fraction from $0.6$ to $1.0$ and frequency from 1 to 10 are reasonable.

**`lambda_l1` and `lambda_l2`**
These are regularization penalties on leaf values. Because the response signal is weak, broad logarithmic ranges are useful:

$$\lambda_1 \in [10^{-8}, 100), \lambda_2 \in [10^{-3}, 100).$$

**`min_gain_to_split`**
A split is permitted only when its estimated loss improvement exceeds this threshold. Search zero together with a logarithmic positive range such as $10^{-5}$ to 1.

**`max_bin`**
LightGBM discretizes continuous features into histogram bins. Values such as 63, 127, 255, and 511 provide a useful discrete search. More bins can capture finer thresholds but require more computation and may expose small unstable regions.

### 13.5 LightGBM-specific safeguards

- Set `boost_from_average=False` because the global effect is supplied explicitly.
- Use early stopping on the exact pairwise validation log loss.
- Keep the same outer and inner folds as all competing algorithms.
- Record the number of boosting iterations selected in every fold.
- Flag a model whose performance relies on one exceptionally large iteration count or one seed.
- Inspect leaf sizes in **minority-arm buyers**, not only total pairwise buyers.
- Calibrate and shrink the raw log-risk-ratio predictions using inner out-of-fold predictions.
- Clip only for numerical safety during log-loss computation; do not use arbitrary probability clipping as a substitute for calibration.

## 14. Pairwise XGBoost models

### 14.1 Relation to the LightGBM formulation

XGBoost can fit the same exact pairwise likelihood. It calls a row-level starting margin `base_margin`. For the training and validation pairwise buyers, supply

$$\text{base\_margin}_i = \log \frac{\pi_{i,t}}{\pi_{i,C}} + \acute{\ell}_t.$$

The boosted trees learn a residual $F_t(x)$, and the final log risk ratio is

$$\widehat{\log RR}_{tC}(x) = \acute{\ell}_t + F_t(x).$$

The propensity offset belongs in the pairwise probability but not in the risk ratio itself.

### 14.2 Native XGBoost implementation pattern

```
import numpy as np
import xgboost as xgb
```

```python
def fit_xgb_pairwise_offset(
    X_train,
    arm_train: np.ndarray,
    pi_train: np.ndarray,
    X_valid,
    arm_valid: np.ndarray,
    pi_valid: np.ndarray,
    *,
    contrast_arm: int,
    params: dict,
    num_boost_round: int = 5000,
    early_stopping_rounds: int = 150,
):
    train_mask = np.isin(arm_train, [contrast_arm, 1])
    valid_mask = np.isin(arm_valid, [contrast_arm, 1])

    y_train = (arm_train[train_mask] == contrast_arm).astype(float)
    y_valid = (arm_valid[valid_mask] == contrast_arm).astype(float)

    off_train = (
        np.log(pi_train[train_mask, contrast_arm])
        - np.log(pi_train[train_mask, 1])
    )
    off_valid = (
        np.log(pi_valid[valid_mask, contrast_arm])
        - np.log(pi_valid[valid_mask, 1])
    )

    global_log_rr = fit_global_pairwise_log_rr(
        arm_train[train_mask],
        y_train,
        pi_train[train_mask, contrast_arm],
        pi_train[train_mask, 1],
    )

    dtrain = xgb.DMatrix(
        X_train.iloc[np.flatnonzero(train_mask)],
        label=y_train,
        base_margin=off_train + global_log_rr,
        enable_categorical=True,
    )
    dvalid = xgb.DMatrix(
        X_valid.iloc[np.flatnonzero(valid_mask)],
        label=y_valid,
        base_margin=off_valid + global_log_rr,
        enable_categorical=True,
    )

    complete_params = {
        "objective": "binary:logistic",
        "eval_metric": "logloss",
        "tree_method": "hist",
        "eta": 0.03,
        "max_depth": 4,
        "min_child_weight": 20.0,
        "subsample": 0.8,
        "colsample_bytree": 0.8,
        "gamma": 0.0,
        "reg_alpha": 0.0,
        "reg_lambda": 10.0,
        "scale_pos_weight": 1.0,
        "seed": 2026,
        "nthread": 1,
        **params,
    }

    booster = xgb.train(
        complete_params,
```

```
        dtrain,
        num_boost_round=num_boost_round,
        evals=[(dvalid, "valid")],
        early_stopping_rounds=early_stopping_rounds,
        verbose_eval=False,
    )
    return booster, global_log_rr


def predict_xgb_log_rr(booster, global_log_rr: float, X) -> np.ndarray:
    # A zero base margin asks XGBoost for the tree residual alone.
    dtest = xgb.DMatrix(
        X,
        base_margin=np.zeros(len(X), dtype=float),
        enable_categorical=True,
    )
    residual = booster.predict(
        dtest,
        output_margin=True,
        iteration_range=(0, int(booster.best_iteration) + 1),
    )
    return global_log_rr + np.asarray(residual, dtype=float)
```

When native `xgb.train` early stopping is used, prediction must be limited to the selected iteration range. Otherwise later trees can be included accidentally.

## 14.3 Important XGBoost hyperparameters

**`eta`**

`eta` is XGBoost's learning rate. Search approximately $0.005$ to $0.10$ on a log scale.

**`max_depth` or `max_leaves`**

With `grow_policy="depthwise"`, tune `max_depth` from about 2 to 7. With `grow_policy="lossguide"`, tune `max_leaves`, for example 8 to 64. Do not tune both as unconstrained independent complexity controls without understanding the growth policy.

**`min_child_weight`**

This parameter requires a minimum sum of Hessians in a child. It is analogous to an information-based minimum leaf size. A broad logarithmic range such as 1 to 100 is useful for pairwise logistic models.

**`gamma`**

`gamma` is the minimum loss reduction required for a split. Include exact zero and a positive logarithmic range such as $10^{-6}$ to 10.

**`subsample`**

This is the row fraction sampled for each tree. Search $0.6$ to $1.0$.

**`colsample_bytree`**

This is the feature fraction sampled for each tree. Search $0.5$ to $1.0$.

**`reg_alpha` and `reg_lambda`**

These are L1 and L2 regularization of leaf weights. Recommended broad ranges are

$$\text{reg\_alpha} \in [10^{-8}, 100), \text{reg\_lambda} \in [10^{-3}, 100).$$

**`max_delta_step`**

This limits the size of a leaf-weight update. Values such as 0, 1, 2, and 5 can be tested. A positive limit can stabilize severe imbalance, although the model should still keep `scale_pos_weight=1` because the imbalance represents the experimental posterior.

**`max_bin`**

For the histogram algorithm, test values such as 64, 128, 256, and 512.

### 14.4 Why `scale_pos_weight` should remain one

For conventional imbalanced classification, `scale_pos_weight` is sometimes increased to emphasize the minority class. Here the minority frequency is not an arbitrary nuisance. It is partly determined by the randomized exposure probabilities and the price effect.

Changing `scale_pos_weight` changes the effective training distribution. Unless the resulting prior shift is exactly reversed, the predicted probabilities no longer estimate the desired pairwise buyer posterior. Keep

```
"scale_pos_weight": 1.0
```

and use proper offsets, likelihoods, and calibration instead.

### 14.5 Comparing LightGBM and XGBoost

The algorithms differ in tree growth, histogram construction, regularization details, and missing-value handling. There is no theoretical reason to declare one universally better for elasticity. They should be compared using:

- identical folds;
- the same eligible feature set;
- separate, similarly budgeted Optuna studies;
- the same pairwise likelihood objective;
- the same cross-fitted calibration procedure;
- paired uncertainty estimates for performance differences.

Algorithm diversity can be valuable for an ensemble only when each component independently passes calibration and stability gates.

## 15. Random Forest approaches

### 15.1 What a Random Forest is

A Random Forest is an ensemble of independently randomized decision trees. Each tree is fitted to a bootstrap sample and considers a random subset of features at each split. Predictions are averaged across trees.

Unlike boosting, a Random Forest does not build trees sequentially to reduce a differentiable loss. It therefore does not naturally incorporate a row-level logit offset. Two workable approaches are:

1. pairwise classification after inverse-propensity weighting;
2. scalar elasticity estimation through moment inversion.

Random Forest should be treated as an algorithmically different robustness benchmark, not as the automatic default.

### 15.2 Pairwise inverse-propensity weighted Random Forest

For pairwise buyers in arm $t$ or control, assign weight

$$w_i = \frac{1}{\pi_{i,T_i}}.$$

After weighting, the positive-class probability is

$$p_w(x) = \frac{q_t(x)}{q_t(x) + q_C(x)}.$$

Therefore,

$$\log RR_{tC}(x) = logit\ p_w(x).$$

A basic implementation is:

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier



def fit_rf_pairwise_ipw(
    X,
```

```
    arm: np.ndarray,
    pi: np.ndarray,
    *,
    contrast_arm: int,
    params: dict,
):
    mask = np.isin(arm, [contrast_arm, 1])
    y = (arm[mask] == contrast_arm).astype(int)
    assigned_pi = pi[mask, arm[mask]]
    weight = 1.0 / assigned_pi
    weight /= weight.mean()

    model = RandomForestClassifier(
        n_estimators=1000,
        criterion="log_loss",
        max_depth=8,
        min_samples_leaf=0.02,
        max_features=0.7,
        bootstrap=True,
        max_samples=0.8,
        ccp_alpha=0.0,
        class_weight=None,
        n_jobs=-1,
        random_state=2026,
        **params,
    )
    model.fit(
        X.iloc[np.flatnonzero(mask)],
        y,
        sample_weight=weight,
    )
    return model



def stable_logit(p: np.ndarray, eps: float = 1e-6) -> np.ndarray:
    p = np.clip(np.asarray(p, dtype=float), eps, 1.0 - eps)
    return np.log(p) - np.log1p(-p)



def predict_rf_log_rr(model, X) -> np.ndarray:
    weighted_probability = model.predict_proba(X)[:, 1]
    return stable_logit(weighted_probability)
```

The clipping in `stable_logit` is a numerical safeguard. It does not make poorly calibrated probabilities valid.

## 15.3 Why Random Forest needs strong smoothing

A high-price pair may contain relatively few high-arm buyers. Small leaves can then have probabilities of 0 or 1, leading to infinite log risk ratios before clipping.

The main controls are:

- `min_samples_leaf`;
- `max_depth`;
- `max_features`;
- `max_samples`;
- `ccp_alpha`;
- out-of-fold affine calibration.

The number of trees mainly controls Monte Carlo noise once it is sufficiently large. It is not the primary overfitting control. Fixing approximately 1,000 trees is usually more efficient than spending the Optuna budget on fine distinctions among 600, 800, and 1,200 trees.

## 15.4 Random Forest hyperparameter search

A suitable search includes:

- `criterion`: `"log_loss"` or `"gini"`;

- `max_depth`: integers from 4 to 16;
- `min_samples_leaf`: a dynamic integer range or a fraction from roughly 0.005 to 0.15;
- `max_features`: `"sqrt"`, `"log2"`, or a fraction from 0.2 to 1.0;
- `max_samples`: 0.5 to 1.0;
- `min_impurity_decrease`: zero or $10^{-8}$ to $10^{-3}$;
- `ccp_alpha`: zero or $10^{-8}$ to $10^{-2}$.

Use `class_weight=None`. Combining inverse-propensity weights with class balancing would change the target again.

## 15.5 Random-Forest moment inversion

The scalar elasticity model supplies another route that avoids treating an invented individual elasticity as a regression label.

Let the centered log-price dose for arm $t$ be

$$d_t = z_t - z_C.$$

For each buyer, observe only

$$d_{T_i}.$$

Weight buyer $i$ by $1/\pi_{i,T_i}$. Under the scalar model, the conditional weighted mean of the observed dose is

$$\mu(x) = \sum_t s_t(x) d_t,$$

where

$$s_t(x) = \frac{\exp\left[-\beta(x) d_t\right)}{\sum_j \exp\left[-\beta(x) d_j\right)}.$$

The function $\mu(x) = M(\beta(x))$ is monotone for fixed doses. Therefore:

1. fit a weighted `RandomForestRegressor` to predict $d_{T_i}$ from $X_i$;
2. interpret its prediction as $\hat{\mu}(x)$;
3. numerically invert $M(\beta) = \hat{\mu}$ by bisection;
4. obtain $\hat{\beta}(x)$ and convert it to both log risk ratios.

```
import numpy as np
from scipy.special import logsumexp
from sklearn.ensemble import RandomForestRegressor



def softmax_mean_dose(beta: np.ndarray, doses: np.ndarray) -> np.ndarray:
    # beta: shape (n,); doses: shape (n, 3), centered at control.
    logits = -beta[:, None] * doses
    probs = np.exp(logits - logsumexp(logits, axis=1, keepdims=True))
    return np.sum(probs * doses, axis=1)



def invert_mean_dose(
    target_mu: np.ndarray,
    doses: np.ndarray,
    beta_max: float = 30.0,
    iterations: int = 60,
) -> np.ndarray:
    low = np.zeros(len(target_mu), dtype=float)
    high = np.full(len(target_mu), beta_max, dtype=float)

    mu_low = softmax_mean_dose(low, doses)
    mu_high = softmax_mean_dose(high, doses)
    target = np.clip(
        target_mu,
        np.minimum(mu_low, mu_high),
```

```
        np.maximum(mu_low, mu_high),
    )

    for _ in range(iterations):
        mid = 0.5 * (low + high)
        mu_mid = softmax_mean_dose(mid, doses)
        move_low = mu_mid > target
        low = np.where(move_low, mid, low)
        high = np.where(move_low, high, mid)

    return 0.5 * (low + high)
```

This is a useful scalar benchmark because it uses Random Forest without pretending that true individual elasticity is observed. Its weaknesses are:

- it estimates a conditional moment rather than the full likelihood;
- inversion can amplify noise near flat parts of $M$;
- predictions can hit the numerical boundaries;
- probability calibration remains indirect;
- pairwise likelihood models are usually easier to evaluate with a proper score.

# 16. Scalar custom objectives for LightGBM and XGBoost

## 16.1 Why use a custom objective?

The pairwise approach estimates two functions, $\log RR_{HC}(x)$ and $\log RR_{LC}(x)$. A scalar model estimates one nonnegative function $\beta(x)$ and imposes local constant elasticity:

$$\log RR_{tC}(x) = -\beta(x) d_t.$$

This stronger structural assumption reduces variance and automatically links the high and low responses. LightGBM and XGBoost do not include this buyer likelihood as a built-in objective, but both permit custom gradients and Hessians.

## 16.2 Raw score and positivity transformation

A tree ensemble naturally produces an unconstrained real score $f(x)$. Define

$$\beta(x) = softplus(f(x)) = \log\left(1 + e^{f(x)}\right).$$

Softplus is positive and differentiable. Its derivatives are

$$\frac{d\beta}{df} = \sigma(f),$$

and

$$\frac{d^2\beta}{df^2} = \sigma(f)\left[1 - \sigma(f)\right).$$

## 16.3 Deriving the gradient

For buyer $i$, define

$$r_{i,t} = \frac{\pi_{i,t} e^{-\beta_i z_{i,t}}}{\sum_j \pi_{i,j} e^{-\beta_i z_{i,j}}}.$$

Let

$$\mu_i = \sum_t r_{i,t} z_{i,t}$$

be the posterior mean dose. The derivative of the buyer negative log-likelihood with respect to $\beta_i$ is

$$\frac{\partial L_i}{\partial \beta_i} = z_{i,T_i} - \mu_i.$$

By the chain rule,

$$\frac{\partial L_i}{\partial f_i} = \left[ z_{i,T_i} - \mu_i \right) \sigma\left(f_i\right).$$

The chain rule states that the derivative of a composite function is the product of its outer and inner derivatives.

## 16.4 Exact and Fisher Hessians

Let

$$Var_{r_i}\left(z_i\right) = \sum_t r_{i,t} z_{i,t}^2 - \mu_i^2.$$

The exact second derivative with respect to the raw score is

$$\frac{\partial^2 L_i}{\partial f_i^2} = Var_{r_i}\left(z_i\right)\sigma\left(f_i\right)^2 + \left[ z_{i,T_i} - \mu_i \right)\sigma\left(f_i\right)\left[1 - \sigma\left(f_i\right)\right).$$

The second term can be negative. Tree-boosting implementations generally behave more reliably with a nonnegative curvature approximation. Use the Fisher or Gauss–Newton Hessian

$$h_i^F = Var_{r_i}\left(z_i\right)\sigma\left(f_i\right)^2 + \varepsilon,$$

where $\varepsilon > 0$ is a small numerical floor.

This is not the exact raw-score Hessian. It is an intentional positive approximation.

## 16.5 Reusable NumPy objective functions

```
from __future__ import annotations

import numpy as np
from scipy.special import expit, logsumexp



def softplus(x: np.ndarray) -> np.ndarray:
    return np.logaddexp(0.0, np.asarray(x, dtype=float))



def scalar_flip_nll_rows(
    beta: np.ndarray,
    arm: np.ndarray,
    pi: np.ndarray,
    z: np.ndarray,
) -> np.ndarray:
    beta = np.asarray(beta, dtype=float).reshape(-1)
    arm = np.asarray(arm, dtype=int).reshape(-1)
    logits = np.log(pi) - beta[:, None] * z
    return (
        logsumexp(logits, axis=1)
        - logits[np.arange(len(arm)), arm]
    )



def scalar_flip_grad_hess(
    raw_score: np.ndarray,
    arm: np.ndarray,
    pi: np.ndarray,
    z: np.ndarray,
    sample_weight: np.ndarray | None = None,
    hessian_floor: float = 1e-8,
) -> tuple[np.ndarray, np.ndarray]:
    f = np.asarray(raw_score, dtype=float).reshape(-1)
    arm = np.asarray(arm, dtype=int).reshape(-1)
```

```
    beta = softplus(f)
    d_beta = expit(f)

    logits = np.log(pi) - beta[:, None] * z
    posterior = np.exp(logits - logsumexp(logits, axis=1, keepdims=True))

    mean_z = np.sum(posterior * z, axis=1)
    var_z = np.maximum(
        np.sum(posterior * z * z, axis=1) - mean_z * mean_z,
        0.0,
    )

    grad_beta = z[np.arange(len(arm)), arm] - mean_z
    grad = grad_beta * d_beta
    hess = var_z * d_beta * d_beta

    if sample_weight is not None:
        weight = np.asarray(sample_weight, dtype=float).reshape(-1)
        grad *= weight
        hess *= weight

    return grad, np.maximum(hess, hessian_floor)
```

Every custom objective should be checked against finite differences before it is trusted.

## 16.6 Finite-difference gradient test

```
def finite_difference_gradient(loss_fn, raw, step=1e-6):
    raw = np.asarray(raw, dtype=float)
    result = np.empty_like(raw)
    for j in range(len(raw)):
        plus = raw.copy()
        minus = raw.copy()
        plus[j] += step
        minus[j] -= step
        result[j] = (loss_fn(plus) - loss_fn(minus)) / (2.0 * step)
    return result


# Example audit on real-shaped arrays, not a performance simulation.
def check_gradient(raw, arm, pi, z):
    def total_loss(f):
        return scalar_flip_nll_rows(softplus(f), arm, pi, z).sum()

    analytical, _ = scalar_flip_grad_hess(raw, arm, pi, z)
    numerical = finite_difference_gradient(total_loss, raw)
    np.testing.assert_allclose(analytical, numerical, rtol=1e-5, atol=1e-6)
```

This test validates implementation algebra. It does not validate the scalar-elasticity assumption on customers.

## 16.7 Initialization at the global elasticity

Fit the global scalar $\hat{\beta}$ inside the training fold. Convert it to a raw score using the inverse softplus:

$$f_0 = \log\left(e^{\hat{\beta}} - 1\right).$$

Supply $f_0$ as the starting margin. Trees then learn residual heterogeneity around the global elasticity. This is preferable to initializing at zero, which corresponds to $\beta = \log 2$ under softplus rather than to no effect.

## 16.8 Custom-objective hyperparameters

Because the scalar model is heavily structured, search shallower trees than in the pairwise models:

### XGBoost scalar model

- `eta`: $0.005$ to $0.05$;

- `max_depth`: 1 to 4;
- `min_child_weight`: $10^{-4}$ to 1 on a log scale, because the custom Fisher Hessians are numerically small;
- `subsample`: 0.6 to 1.0;
- `colsample_bytree`: 0.5 to 1.0;
- `gamma`: $10^{-7}$ to 1, including zero;
- `reg_alpha`: $10^{-8}$ to 100;
- `reg_lambda`: $10^{-3}$ to 100;
- `max_bin`: 64, 128, or 256.

### LightGBM scalar model

- `learning_rate`: $0.005$ to $0.05$;
- `max_depth`: 1 to 4;
- `num_leaves`: 2 to $min\left(15, 2^{\text{depth}}\right)$;
- `min_data_in_leaf`: approximately 1% to 20% of buyer rows, with an absolute floor;
- `min_sum_hessian_in_leaf`: $10^{-5}$ to 0.5;
- `feature_fraction`: 0.5 to 1.0;
- `bagging_fraction`: 0.6 to 1.0;
- L1 and L2 regularization on logarithmic scales.

The custom Hessian scale differs from ordinary binary logistic curvature, so ranges copied directly from a classification tutorial may be inappropriate.

# 17. Constrained multiclass and geometric models

## 17.1 Direct three-probability model

The original Bayes-flip proposal predicts

$$r(x)=\left[r_H(x), r_C(x), r_L(x)\right)$$

inside the probability simplex

$$\Delta^2=\left\{r \in R^3 : r_t \geq 0, r_H+r_C+r_L=1\right\}.$$

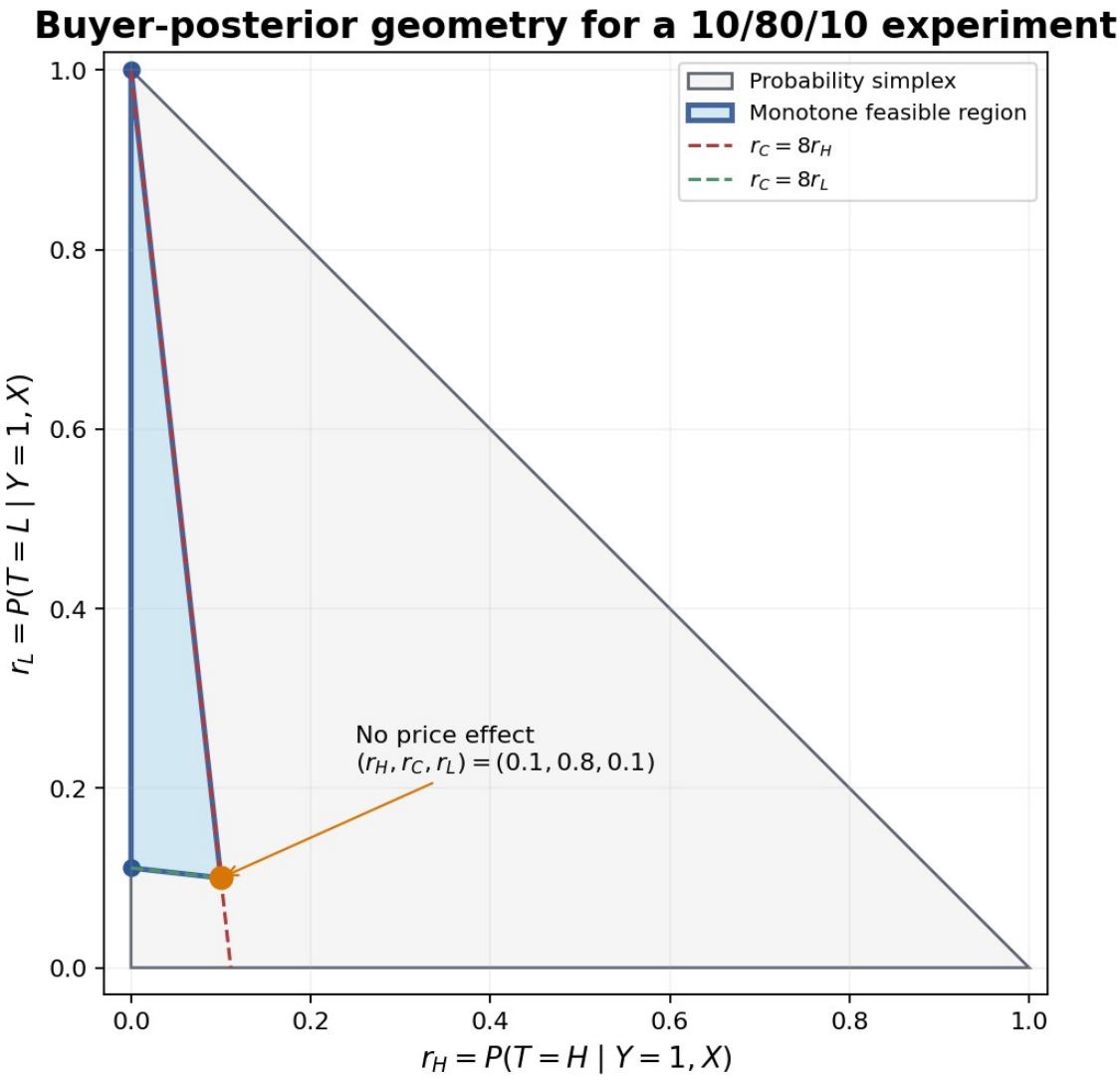
The monotone-demand region adds

$$\frac{r_H}{\pi_H} \leq \frac{r_C}{\pi_C} \leq \frac{r_L}{\pi_L}.$$

For $\pi=(0.1, 0.8, 0.1)$, this becomes

$$r_H \leq \frac{r_C}{8} \leq r_L.$$

The feasible set is a triangle inside the larger probability simplex.




*The monotone feasible region within the buyer-probability simplex*

## 17.2 Two-gap exact parameterization

Set

$$u(x) \geq 0, v(x) \geq 0.$$

Define

$$r(x) = softmax\left(\log \pi(x) + \left[-u(x), 0, v(x)\right]\right).$$

Then

$$\log RR_{HC}(x) = -u(x) \leq 0,$$

and

$$\log RR_{LC}(x) = v(x) \geq 0.$$

This parameterization covers every strictly positive buyer posterior that satisfies the monotonicity inequalities. Use softplus outputs,

$$u(x) = softplus(a(x)), v(x) = softplus(b(x)),$$

to ensure nonnegativity.

The two-gap model is usually cleaner in a differentiable multi-output framework. Standard Random Forest does not directly optimize this coupled multiclass likelihood, while LightGBM and XGBoost custom objectives are simplest when the row Hessian is scalar or diagonal.

## 17.3 Barycentric triangle parameterization

For fixed $10/80/10$ propensities, the closed feasible triangle has vertices

$$V_1=(0,0,1),$$

$$V_2=\left(0,\frac{8}{9},\frac{1}{9}\right),$$

$$V_3=(0.1,0.8,0.1).$$

Predict nonnegative weights $w_1, w_2, w_3$ with

$$w_1+w_2+w_3=1,$$

for example by a softmax. Then set

$$r=w_1V_1+w_2V_2+w_3V_3.$$

This guarantees feasibility. Its disadvantages are that the weights are less directly interpretable than log risk ratios and that the geometry changes with row-level propensities.

## 17.4 One-number plus cutoffs: why an ordinary ordinal model is weak

A conventional ordinal classifier assumes that the observed class arises by thresholding one latent score. That mechanism is not appropriate here because the price arm was randomized; a customer's arm is not an ordinal behavioral outcome.

A one-number model becomes meaningful when the number is explicitly defined as elasticity:

$$\beta(x)\geq 0,$$

and the class probabilities are generated by the known softmax likelihood. The thresholds are replaced by known log-price doses and assignment propensities. This is the scalar model described earlier.

## 17.5 Pairwise classifiers versus unrestricted one-versus-rest classifiers

Two pairwise models with control estimate the exact quantities required:

$$\log RR_{HC}, \log RR_{LC}.$$

Three one-versus-rest classifiers estimate different conditional probabilities. Their outputs need not sum to one and do not map cleanly to the risk ratios without extra assumptions. Pairwise-with-control is therefore preferable.

## 17.6 Post-processing by projection

Suppose an unrestricted model produces a buyer posterior $\tilde{r}$ outside the feasible triangle. Projection solves

$$\hat{r}=\arg\min_{r\in F} D(r,\tilde{r}),$$

where $F$ is the feasible region and $D$ is a distance or divergence.

Choices include:

- Euclidean distance after Brier-score training;
- Kullback–Leibler divergence after cross-entropy training;
- isotonic projection on the three arm-specific normalized risks $r_t/\pi_t$.

Projection is a valid safety layer, but it does not recover information lost to a poorly specified model. Always report:

- the fraction of rows changed;
- the mean and upper-tail projection distance;
- changes in validation likelihood;
- whether the projected model collapses large regions to the boundary.

A high violation or projection rate is evidence against the unconstrained model, not merely a cosmetic inconvenience.

### 17.7 Preprocessing outcomes to force monotonicity is invalid

Deleting, duplicating, relabeling, or modifying observed buyers to manufacture monotone arm frequencies changes the randomized experiment. It can bias the relative-risk target and invalidate likelihood-based uncertainty.

Valid preprocessing includes:

- correcting documented data errors;
- applying a pre-specified eligibility definition;
- resolving duplicate quote records according to protocol;
- excluding post-treatment variables from features;
- encoding missingness and categories inside training folds.

It does not include editing outcomes to fit economic beliefs.

### 17.8 Oversampling and prior correction

If buyers from arm $t$ are sampled with probability $s_t$, the fitted posterior without correction is proportional to $s_t r_t(x)$. The original posterior is recovered by

$$r_t(x) = \frac{\tilde{r}_t(x)/s_t}{\sum_j \tilde{r}_j(x)/s_j}.$$

The equivalent logit correction subtracts $\log s_t$ from the sampled class score. Oversampling may help engineering throughput, but it creates no statistical information and should rarely be needed for a buyer sample of manageable size.

## 18. Calibration, shrinkage, and monotonicity

### 18.1 Discrimination is not calibration

A model has good **discrimination** if it ranks observations in a useful order. It is **calibrated** if the numerical magnitude of its probabilities or risk ratios agrees with observed randomized evidence.

A model can rank customers correctly but exaggerate every effect by a factor of three. Such a model may have useful AUC yet produce damaging price decisions.

### 18.2 Affine calibration of log risk ratios

Let $g_i$ be a raw out-of-fold predicted log risk ratio for one contrast. Fit

$$g_i^{cal} = a + s(g_i - c), s \geq 0,$$

where:

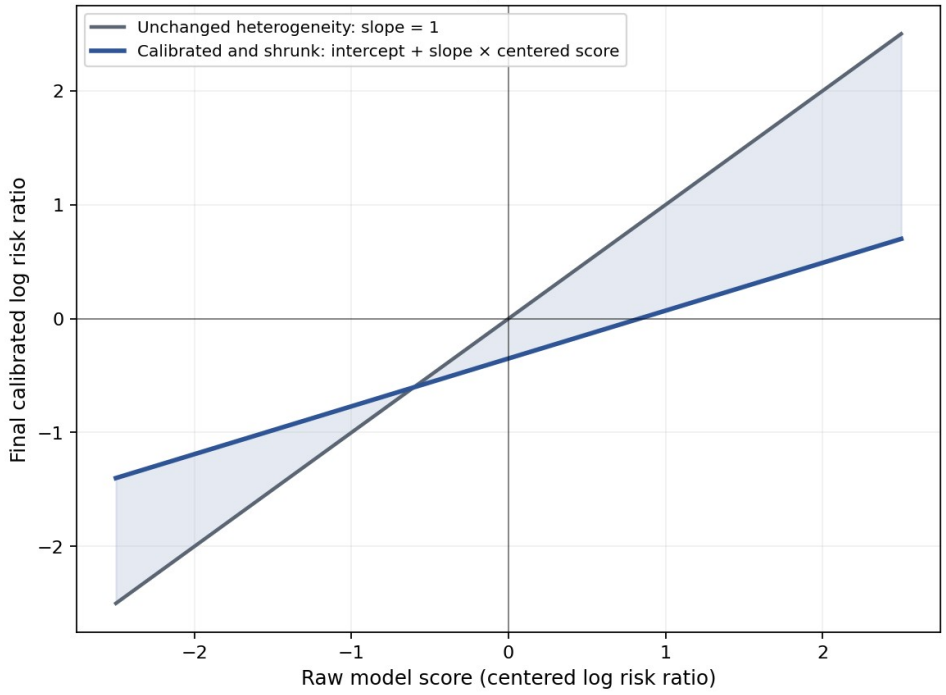
- $a$ is the calibrated average effect;
- $s$ is the heterogeneity slope;
- $c$ is a fixed center, usually the weighted mean of $g_i$ in the calibration data;
- $s = 1$ preserves the raw spread;
- $0 < s < 1$ shrinks heterogeneity;
- $s = 0$ discards the ranking and returns a global effect.

Fit $a$ and $s$ by minimizing the exact pairwise offset log loss:

$$\sum_i \left( \log\left[1 + e^{o_i + g_i^{cal}}\right) - Z_i\left[o_i + g_i^{cal}\right) \right).$$

The slope is parameterized as $s = e^h$ or otherwise constrained to be nonnegative.

**Affine calibration can correct the average effect and shrink unstable heterogeneity**




Schematic only: no empirical data are shown. A fitted slope near zero means the flexible model did not learn reproducible heterogeneity.

*Cross-fitted affine calibration shrinks unreliable heterogeneity*

## 18.3 Why calibration predictions must be out of fold

If the same rows are used to train the base model and its calibrator, overfit predictions appear more informative than they are. The calibrator may preserve excessive spread instead of shrinking it.

Inside one outer training fold:

1. divide the outer-training rows into inner folds;
2. train the base learner on the inner-training part;
3. predict the inner-validation part;
4. concatenate these inner out-of-fold predictions;
5. fit the calibrator on that concatenated set;
6. refit the base learner on all outer-training rows using the chosen hyperparameters;
7. apply the frozen calibrator to the outer-validation predictions.

No outer-validation outcome may influence the calibrator.

## 18.4 Sign constraints

Economic monotonicity implies

$$g_H(x) = \log RR_{HC}(x) \le 0,$$

and

$$g_L(x) = \log RR_{LC}(x) \ge 0.$$

A hard projection is

$$\tilde{g}_H = min(g_H, 0), \tilde{g}_L = max(g_L, 0).$$

A smooth training-time alternative uses softplus:

$$\tilde{g}_H = -\tau \log\left(1 + e^{-g_H/\tau}\right),$$

$$\tilde{g}_L = \tau \log\left(1 + e^{g_L/\tau}\right),$$

where $\tau > 0$ controls smoothness.

Use a hard constraint for final production scores. Keep a diagnostic copy of the unconstrained predictions so that violation frequency remains observable.

### 18.5 Shrinking toward the global scalar anchor

A direct shrinkage formula is

$$\ell_t^{final}(x) = \ell_t^{global} + \lambda_t \left[ \ell_t^{flex}(x) - \ell_t^{global} \right],$$

with

$$0 \le \lambda_t \le 1.$$

Choose $\lambda_t$ using inner out-of-fold likelihood. This is a special case of affine calibration and is easy to explain to governance stakeholders.

### 18.6 Calibration diagnostics

For each contrast, report:

- intercept $a$;
- slope $s$;
- confidence intervals from customer-cluster bootstrap;
- raw and calibrated pairwise NLL;
- sign-violation rate before projection;
- mean absolute change from calibration;
- percentile plots of raw versus calibrated log risk ratio;
- calibration by time period, channel, and tariff version.

A slope near zero means that the flexible model's ranking is not reproducible after accounting for the global effect. A negative unconstrained slope is especially concerning; forcing it positive should not be presented as evidence that the ranking works.

## 19. Measuring elasticity-model quality

### 19.1 Why no single metric is sufficient

The model has several possible failure modes:

- wrong probability ratios;
- correct average effect but noisy individual variation;
- useful ranking but exaggerated magnitude;
- good likelihood but unstable business decisions near a threshold;
- temporal drift;
- apparent value caused by reuse of training outcomes.

Evaluation must therefore combine proper likelihood, calibration, randomized group effects, stability, and policy value.

### 19.2 Buyer multiclass cross-entropy

For buyer $i$, let $\hat{r}_{i,t}$ be the predicted probability of arm $t$. The buyer cross-entropy is

$$CE_B = -\frac{1}{n_B} \sum_{i:Y_i=1} \log \hat{r}_{i,T_i},$$

where $n_B$ is the number of buyers.

This is a **strictly proper scoring rule**: in expectation, the true conditional probability distribution uniquely minimizes the score. It directly penalizes incorrect probability ratios.

### 19.3 Pairwise cross-entropy

For contrast $t/C$, define

$$\hat{p}_{i,tC} = \sigma\left[\log\left(\pi_{i,t}/\pi_{i,C}\right) + \widehat{\log RR}_{tC}\left(X_i\right)\right].$$

The pairwise NLL is

$$NLL_{tC} = \frac{1}{n_{tC}} \sum_{i \in I_{tC}} \left[\log\left(1+e^{\eta_i}\right) - Z_i \eta_i\right),$$

where $\eta_i$ is the pairwise logit and $I_{tC}$ is the set of buyers in the pair.

Use the average of H/C and L/C scores with equal contrast weights when comparing joint systems.

## 19.4 Information gain over the global anchor

Define

$$\Delta_{info} = NLL_{global} - NLL_{flex}.$$

A positive value favors the flexible model. Report

$$1000\,\Delta_{info}$$

as **millinats per buyer**. A nat is the information unit associated with natural logarithms. Millinats make small but potentially real likelihood differences easier to read.

Estimate a paired confidence interval by resampling customers or households and recomputing the average per-row loss difference. The comparison is paired because both models score the same observations.

## 19.5 Why AUC and VUS are secondary

AUC measures ranking. Multiclass VUS is a three-class ranking analogue. Neither requires calibrated numerical probabilities.

Elasticity depends on quantities such as

$$\log \frac{r_H/\pi_H}{r_C/\pi_C}.$$

Class-specific rescaling can substantially alter this ratio while preserving much of the ordering. Conversely, a correct homogeneous elasticity produces little or no cross-customer ranking signal.

Therefore:

- cross-entropy is the primary probability metric;
- AUC or VUS can be reported as secondary diagnostics of heterogeneity ranking;
- neither should select the production elasticity model by itself.

## 19.6 Grouped randomized calibration

Individual elasticity is not directly observed. To assess heterogeneity:

1. generate strictly out-of-fold scores for every eligible quote;
2. rank quotes by the relevant predicted log risk ratio or business score;
3. form a small number of sufficiently large groups;
4. within each group, estimate arm conversion risks using the randomized data;
5. compare experimental group risk ratios with model-implied group risk ratios.

For group $G$,

$$RR_{tC}(G) = \frac{E\left[q_t(X) \mid X \in G\right)}{E\left[q_C(X) \mid X \in G\right)}.$$

This is a ratio of average risks. It is generally **not** equal to the unweighted average of individual $RR_{tC}(X)$ or individual elasticities.

When using shape-only buyer predictions, an exact useful identity is

$$RR_{tC}(G) = E\left[RR_{tC}(X) \mid T=C, Y=1, X \in G\right)$$

under constant control propensity within the group. If $\pi_C(X)$ varies, use the corresponding inverse-propensity weighting.

Large groups are essential. With a small high-price arm, ten deciles may be too noisy. Three to five groups can be more honest.

## 19.7 Experimental group-risk estimators

For group $G$ and arm $t$, an inverse-propensity estimator is

$$\hat{q}_t(G) = \frac{\sum_{i:X_i \in G} 1(T_i = t) Y_i / \pi_{i,t}}{\sum_{i:X_i \in G} 1(T_i = t) / \pi_{i,t}}.$$

Then

$$\widehat{RR}_{tC}(G) = \frac{\hat{q}_t(G)}{\hat{q}_C(G)}.$$

Use cluster bootstrap intervals. Ratios can be unstable when a group has very few purchases, so report numerator and denominator counts and risks together.

## 19.8 Calibration regression

A pairwise calibration regression on held-out buyers is

$$\mathit{logit}\, P(T = t \mid \text{pair buyer}, X) = \log(\pi_t / \pi_C) + a + b\, \widehat{\log RR}_{tC}(X).$$

Interpretation:

- $a = 0$ means the average effect is calibrated;
- $b = 1$ means the predicted heterogeneity scale is calibrated;
- $0 < b < 1$ indicates over-dispersed predictions;
- $b \approx 0$ indicates no validated heterogeneity;
- $b > 1$ indicates under-dispersed predictions;
- $b < 0$ indicates reversed ranking.

The affine calibrator uses this same principle operationally.

## 19.9 Stability metrics

Report:

- rank correlation across outer folds and random seeds;
- overlap of top-target groups;
- distribution of log risk ratios by period;
- performance by tariff version and channel;
- sensitivity to feature subsets;
- sensitivity to repeat-quote handling;
- sensitivity to propensity estimates and compliance exclusions;
- fraction of decisions that change under bootstrap refits.

A high average score with unstable targeting is not enough for personalized pricing.

# 20. Business-policy evaluation without a base-conversion model

## 20.1 From elasticity to a policy

A **policy** is a function

$$d(x) \in \{H, C, L\}$$

that assigns one tested price arm to context $x$.

Given risk-ratio predictions and conditional-on-sale contribution $g_t(x)$, the plug-in policy is

$$\hat{d}(x) = \arg\max_t g_t(x)\widehat{RR}_{tC}(x).$$

This uses no estimated $q_C(x)$.

## 20.2 Inverse-propensity policy value

The randomized experiment permits direct evaluation of a fixed policy. Let the observed reward be

$$R_i = Y_i g_{T_i}(X_i).$$

The inverse-propensity-score estimator is

$$\hat{V}_{IPS}(d) = \frac{1}{N}\sum_{i=1}^{N} \frac{1\{T_i = d(X_i)\}R_i}{\pi_{i,T_i}}.$$

The indicator is one when the experiment happened to assign the same arm that the policy would choose.

The policy predictions must be out of fold. A policy evaluated on the same outcomes used to train it is optimistically biased.

## 20.3 Self-normalized IPS

A stabilized alternative is

$$\hat{V}_{SNIPS}(d) = \frac{\sum_i 1\{T_i = d(X_i)\}R_i/\pi_{i,T_i}}{\sum_i 1\{T_i = d(X_i)\}/\pi_{i,T_i}}.$$

SNIPS can reduce finite-sample variability but introduces small finite-sample bias. Report both when weights vary substantially.

## 20.4 Doubly robust evaluation as an optional side model

Let $m_t(x) = E[R \mid T = t, X = x]$ be an outcome model. A doubly robust value estimator is

$$\hat{V}_{DR}(d) = \frac{1}{N}\sum_i \left[\hat{m}_{d(X_i)}(X_i) + \frac{1\{T_i = d(X_i)\}}{\pi_{i,T_i}}\{R_i - \hat{m}_{T_i}(X_i)\}\right].$$

This is where a base-conversion or reward model can be useful as a side effect: it may reduce policy-value variance. It is not required to define the elasticity model or the arm choice.

The outcome model must also be cross-fitted.

## 20.5 Policy comparators

Every personalized model should be compared with:

- control for everyone;
- high price for everyone;
- low price for everyone;
- the empirically best uniform tested arm;
- the global scalar-elasticity policy;
- simple approved segment rules;
- an abstaining policy that adjusts only high-confidence cases.

The most important comparison is often against the global-elasticity policy, because it already uses quote-specific margins.

## 20.6 Conservative decision rules

For high versus control, define the estimated log contribution advantage

$$\hat{S}_H(x) = \log \frac{g_H(x)}{g_C(x)} + \widehat{\log RR}_{HC}(x).$$

A conservative rule changes price only if a lower confidence bound is positive:

$$LCB\{S_H(x)\} > 0.$$

Individual confidence intervals from one flexible model are difficult. Practical conservative substitutes include:

- agreement across outer-fold refits;
- bootstrap prediction quantiles;
- model-ensemble agreement;
- minimum predicted gain thresholds;
- targeting only segments with randomized group calibration;
- explicit caps on the proportion assigned to each price change.

## 20.7 Portfolio constraints

A constrained policy can be written as

$$\max_d E\left[g_{d(X)}(X)\, q_{d(X)}(X)\right]$$

subject to constraints such as

$$P(d(X) = H) \le \kappa_H,$$
$$P(d(X) = L) \le \kappa_L,$$

and fairness, regulatory, volume, or risk limits.

Because the common $q_C(x)$ no longer cancels from some portfolio-wide constraints, absolute conversion may become relevant for full constrained optimization. A practical elasticity-only deployment can instead rank by relative contribution gain and apply business quotas, then evaluate the resulting policy directly by IPS.

# 21. Optuna optimization in detail

## 21.1 What hyperparameter optimization means

A **model parameter** is learned directly from data during fitting, such as a logistic-regression coefficient or a tree leaf value.

A **hyperparameter** controls the learning process or model complexity, such as tree depth, regularization strength, or learning rate. Hyperparameters are selected using validation data, not the final test data.

Optuna is a framework for searching hyperparameter configurations. A **study** is one optimization run. A **trial** is one proposed hyperparameter configuration and its validation result.

## 21.2 Separate studies rather than one giant conditional study

Create separate studies for:

- H/C penalized logistic;
- L/C penalized logistic;
- H/C LightGBM;
- L/C LightGBM;
- H/C XGBoost;
- L/C XGBoost;
- H/C Random Forest;
- L/C Random Forest;
- scalar LightGBM;
- scalar XGBoost;

- Random-Forest moment model.

Reasons:

- each algorithm has different parameter geometry;
- the H/C and L/C contrasts have different buyer counts and noise;
- study diagnostics are easier to interpret;
- pruning and search budgets can be allocated sensibly;
- one algorithm's large conditional space does not distort another's TPE density estimates.

## 21.3 TPE sampler

The Tree-structured Parzen Estimator, or TPE, uses results from completed trials to construct models of promising and less promising hyperparameter regions. It then proposes configurations expected to improve the objective.

A practical setup is:

```
import optuna

sampler = optuna.samplers.TPESampler(
    seed=2026,
    n_startup_trials=30,
    multivariate=True,
    group=True,
)
```

The first `n_startup_trials` are exploratory. Version-pin Optuna because some advanced sampler options are documented as experimental and can change across releases.

## 21.4 Pruning

A **pruner** stops an unpromising trial before its full computational budget is spent. For boosting, report validation loss at intermediate iteration blocks.

```
pruner = optuna.pruners.MedianPruner(
    n_startup_trials=15,
    n_warmup_steps=1,
    n_min_trials=5,
)
```

Pruning must use inner validation only. Never prune based on outer validation or final holdout performance.

## 21.5 Objective function for one contrast

```
def pairwise_nll(y: np.ndarray, offset: np.ndarray, log_rr: np.ndarray) -> float:
    eta = offset + log_rr
    rows = np.logaddexp(0.0, eta) - y * eta
    return float(rows.mean())
```

One Optuna trial should average this score across the inner folds. Preserve fold-level values for uncertainty and diagnostics.

## 21.6 Joint objective for a two-contrast system

When tuning a shared design or comparing complete systems,

$$J = \frac{1}{2} NLL_{HC} + \frac{1}{2} NLL_{LC}.$$

Optional secondary penalties can reject rather than reward models with:

- excessive sign violations;
- extreme log-risk-ratio tails;
- severe calibration failure;
- unstable fold performance.

Avoid an opaque weighted sum of many metrics. Proper likelihood should remain the primary numerical optimization target, with calibration and stability applied as gates.

## 21.7 Dynamic leaf-size bounds

Let $n_{pair}$ be the number of buyers in the current pair and $\rho_{min}$ the minority-arm fraction. Define

$$L = max\left(20, \left\lceil \frac{10}{\rho_{min}} \right\rceil\right),$$

and

$$U = max\left[L, min\left(0.25\, n_{pair}, \left\lceil \frac{100}{\rho_{min}} \right\rceil\right)\right].$$

Search `min_data_in_leaf` or `min_samples_leaf` between $L$ and $U$, preferably on a logarithmic scale. The constants 10 and 100 express a conservative desired range of minority observations per minimally sized region. They should be reviewed against actual buyer counts.

## 21.8 Search-space functions

```
from __future__ import annotations

import math
import optuna



def dynamic_leaf_bounds(n_pair: int, minority_fraction: float) -> tuple[int, int]:
    if n_pair <= 0 or not 0.0 < minority_fraction <= 0.5:
        raise ValueError("Invalid pair size or minority fraction")
    lower = max(20, math.ceil(10.0 / minority_fraction))
    upper_candidate = min(
        int(0.25 * n_pair),
        math.ceil(100.0 / minority_fraction),
    )
    upper = max(lower, upper_candidate)
    return lower, upper



def suggest_lgb_pairwise(
    trial: optuna.Trial,
    n_pair: int,
    minority_fraction: float,
) -> dict:
    depth = trial.suggest_int("max_depth", 2, 7)
    leaf_low, leaf_high = dynamic_leaf_bounds(n_pair, minority_fraction)
    return {
        "learning_rate": trial.suggest_float(
            "learning_rate", 0.005, 0.10, log=True
        ),
        "max_depth": depth,
        "num_leaves": trial.suggest_int(
            "num_leaves", 4, min(64, 2**depth - 1)
        ),
        "min_data_in_leaf": trial.suggest_int(
            "min_data_in_leaf", leaf_low, leaf_high, log=True
        ),
        "min_sum_hessian_in_leaf": trial.suggest_float(
            "min_sum_hessian_in_leaf", 0.1, 50.0, log=True
        ),
        "feature_fraction": trial.suggest_float(
            "feature_fraction", 0.5, 1.0
        ),
        "bagging_fraction": trial.suggest_float(
            "bagging_fraction", 0.6, 1.0
        ),
        "bagging_freq": trial.suggest_int("bagging_freq", 1, 10),
        "lambda_l1": trial.suggest_float(
            "lambda_l1", 1e-8, 100.0, log=True
        ),
        "lambda_l2": trial.suggest_float(
```

```
            "lambda_l2", 1e-3, 100.0, log=True
        ),
        "min_gain_to_split": trial.suggest_categorical(
            "min_gain_to_split",
            [0.0, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1.0],
        ),
        "max_bin": trial.suggest_categorical(
            "max_bin", [63, 127, 255, 511]
        ),
        "feature_pre_filter": False,
    }


def suggest_xgb_pairwise(trial: optuna.Trial) -> dict:
    grow_policy = trial.suggest_categorical(
        "grow_policy", ["depthwise", "lossguide"]
    )
    params = {
        "eta": trial.suggest_float("eta", 0.005, 0.10, log=True),
        "grow_policy": grow_policy,
        "min_child_weight": trial.suggest_float(
            "min_child_weight", 1.0, 100.0, log=True
        ),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float(
            "colsample_bytree", 0.5, 1.0
        ),
        "gamma": trial.suggest_categorical(
            "gamma", [0.0, 1e-6, 1e-4, 1e-2, 0.1, 1.0, 10.0]
        ),
        "reg_alpha": trial.suggest_float(
            "reg_alpha", 1e-8, 100.0, log=True
        ),
        "reg_lambda": trial.suggest_float(
            "reg_lambda", 1e-3, 100.0, log=True
        ),
        "max_delta_step": trial.suggest_categorical(
            "max_delta_step", [0.0, 1.0, 2.0, 5.0]
        ),
        "max_bin": trial.suggest_categorical(
            "max_bin", [64, 128, 256, 512]
        ),
        "scale_pos_weight": 1.0,
    }
    if grow_policy == "depthwise":
        params["max_depth"] = trial.suggest_int("max_depth", 2, 7)
    else:
        params["max_depth"] = 0
        params["max_leaves"] = trial.suggest_int("max_leaves", 8, 64)
    return params


def suggest_random_forest(
    trial: optuna.Trial,
    n_pair: int,
    minority_fraction: float,
) -> dict:
    leaf_low, leaf_high = dynamic_leaf_bounds(n_pair, minority_fraction)
    max_features_kind = trial.suggest_categorical(
        "max_features_kind", ["sqrt", "log2", "fraction"]
    )
    max_features = (
        trial.suggest_float("max_features_fraction", 0.2, 1.0)
        if max_features_kind == "fraction"
        else max_features_kind
    )
    return {
        "n_estimators": 1000,
        "criterion": trial.suggest_categorical(
            "criterion", ["log_loss", "gini"]
```

```
        ),
        "max_depth": trial.suggest_int("max_depth", 4, 16),
        "min_samples_leaf": trial.suggest_int(
            "min_samples_leaf", leaf_low, leaf_high, log=True
        ),
        "max_features": max_features,
        "max_samples": trial.suggest_float("max_samples", 0.5, 1.0),
        "min_impurity_decrease": trial.suggest_categorical(
            "min_impurity_decrease",
            [0.0, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3],
        ),
        "ccp_alpha": trial.suggest_categorical(
            "ccp_alpha", [0.0, 1e-8, 1e-6, 1e-4, 1e-3, 1e-2]
        ),
        "class_weight": None,
    }
```

## 21.9 Trial budgets

Suggested starting budgets, subject to data size and compute:

- penalized logistic: 50–100 trials per contrast;
- LightGBM: 150–300 trials per contrast;
- XGBoost: 150–300 trials per contrast;
- Random Forest: 100–200 trials per contrast;
- scalar custom boosting: 100–200 trials per algorithm.

Budgets should be increased only when:

- optimization curves still improve materially;
- repeated studies select similar regions;
- the nested validation design is preserved;
- compute does not force shortcuts in leakage control.

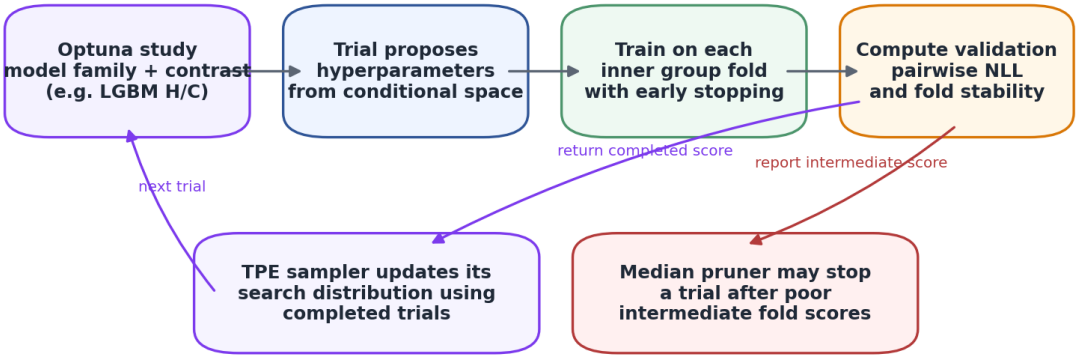
More trials are not automatically better. Repeated tuning against the same inner folds can itself overfit the validation design.

## 21.10 Study reproducibility

Store:

- Optuna and library versions;
- study database;
- sampler and pruner configuration;
- random seed;
- feature manifest hash;
- data hash or immutable snapshot identifier;
- fold assignments;
- every trial's parameters, intermediate values, and failure reason;
- selected iteration counts;
- calibration parameters fitted after tuning.

**Optuna optimizes a proper buyer-side likelihood—not AUC or in-sample profit**




*Run separate studies for H/C, L/C, and scalar models. Persist studies in a database so every trial and failure is auditable.*

*Nested optimization and evaluation workflow*

# 22. End-to-end real-data workflow

## 22.1 Stage A: freeze the estimand and protocol

Before code is run, write down:

- the eligible quote population;
- the conversion window;
- whether the primary treatment is assignment or displayed dose;
- the customer/household clustering unit;
- the definition of conditional-on-sale contribution $g_t(x)$;
- which price arms are permitted operationally;
- the intended deployment population and time horizon;
- primary and secondary evaluation metrics;
- exclusion and missing-data rules.

Changing these after observing model results creates researcher degrees of freedom.

## 22.2 Stage B: build a feature manifest

For every candidate feature, record:

- name and data type;
- business meaning;
- source system;
- timestamp relative to randomization;
- missingness semantics;
- whether it can be known at scoring time;
- potential regulatory sensitivity;
- expected monotonic or nonlinear behavior;
- owner and data-quality checks.

Classify every feature as:

- approved pre-treatment;
- post-treatment and prohibited;
- uncertain and requiring investigation;

- identifier used only for splitting or auditing.

## 22.3 Stage C: validate the experiment table

```
REQUIRED_COLUMNS = {
    "quote_id",
    "customer_id",
    "quote_timestamp",
    "eligible",
    "arm",
    "pi_H",
    "pi_C",
    "pi_L",
    "assigned_multiplier",
    "base_premium",
    "displayed_premium",
    "purchase",
    "tariff_version",
}


def validate_experiment_frame(df):
    missing = REQUIRED_COLUMNS - set(df.columns)
    if missing:
        raise ValueError(f"Missing required columns: {sorted(missing)}")

    if not df["quote_id"].is_unique:
        raise ValueError("quote_id must be unique after documented deduplication")

    if not set(df["arm"].dropna().unique()) <= {"H", "C", "L"}:
        raise ValueError("arm must contain only H, C, or L")

    if not set(df["purchase"].dropna().unique()) <= {0, 1}:
        raise ValueError("purchase must be binary")

    propensity = df[["pi_H", "pi_C", "pi_L"]].to_numpy(float)
    if np.any(~np.isfinite(propensity)) or np.any(propensity <= 0.0):
        raise ValueError("Every logged propensity must be finite and positive")
    if not np.allclose(propensity.sum(axis=1), 1.0, atol=1e-8):
        raise ValueError("Propensities must sum to one per row")
```

The code should fail closed. It must not silently replace missing propensities with nominal values unless the experiment documentation proves they were constant.

## 22.4 Stage D: perform model-free experiment checks

Produce:

- quote and buyer counts by arm;
- observed versus expected assignment counts;
- conversion rate and confidence interval by arm;
- global H/C and L/C risk ratios;
- standardized covariate balance by arm;
- assignment shares by date, channel, tariff version, and randomization block;
- missingness by arm;
- treatment-compliance summaries;
- repeat-customer patterns;
- exclusion flow diagram.

Stop when assignment or outcome integrity is doubtful.

## 22.5 Stage E: freeze splits

A robust design has:

1. a final chronological holdout containing the latest period;

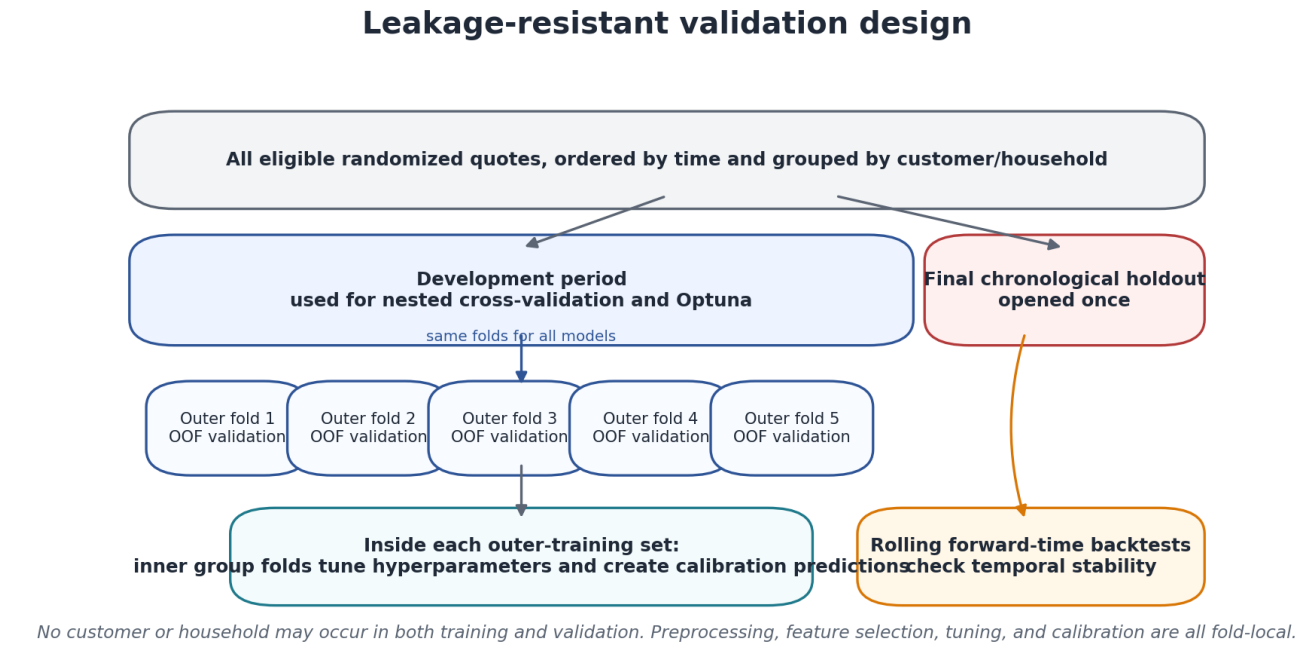
2. customer- or household-disjoint outer folds within the development period;
3. customer-disjoint inner folds for Optuna and calibration;
4. identical fold identifiers for every candidate model.

```
from sklearn.model_selection import StratifiedGroupKFold

inner_cv = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=2026,
)

# y_stratify is buyer arm for buyer-only tuning.
# groups is customer_id or household_id.
for train_index, valid_index in inner_cv.split(
    X_buyers,
    y_stratify,
    groups=buyer_customer_id,
):
    ...
```

The final chronological holdout must remain untouched until the complete model family, hyperparameters, calibration method, and decision rule have been selected.




*Customer-disjoint nested validation with a final time holdout*

## 22.6 Stage F: fit mandatory anchors

Fit and record:

- no-effect buyer model $r = \pi$;
- unrestricted global two-gap model;
- global scalar elasticity model;
- global-elasticity business policy.

If the global effect is not stable across time and major operational segments, personalized elasticity should not proceed.

## 22.7 Stage G: tune model families

For every outer fold and contrast:

1. use outer-training buyers only;
2. run a separate Optuna study across inner folds;
3. select hyperparameters by mean inner pairwise NLL;

4. create inner out-of-fold raw predictions with the selected configuration;
5. fit the affine calibrator on those predictions;
6. refit the base model on all outer-training buyers;
7. score outer-validation rows;
8. apply the frozen calibrator and sign projection;
9. store all predictions and metadata.

This yields one out-of-fold prediction per development quote without training on that quote's outcome.

### 22.8 Stage H: compare complete systems

For each system, calculate:

- H/C and L/C pairwise NLL;
- equal-contrast NLL;
- buyer multiclass NLL after coherent reconstruction;
- information gain over global anchor;
- calibration intercept and slope;
- grouped randomized risk ratios;
- sign-violation and projection rates;
- temporal and seed stability;
- targeting and policy-value metrics;
- complexity and runtime.

Use paired customer-cluster intervals for model differences.

### 22.9 Stage I: refit and evaluate the final holdout

After the complete specification is frozen:

- rerun Optuna only according to the pre-specified development procedure, or use the aggregate selected hyperparameters;
- fit calibration using development-only out-of-fold predictions;
- refit the base learners on all development buyers;
- score the chronological holdout exactly once;
- report every pre-specified metric, including unfavorable results;
- do not return to tuning after viewing the holdout.

### 22.10 Stage J: shadow deployment

Before price decisions affect customers:

- score live quotes without changing their price;
- monitor feature availability and latency;
- compare predicted score distributions with development data;
- log recommended arm, actual arm, propensities, model version, and reason codes;
- verify contribution calculations;
- define rollback and abstention behavior.

Then use a fresh randomized validation phase rather than immediately making the learned policy deterministic everywhere.

## 23. Feature engineering for elasticity heterogeneity

### 23.1 What makes a feature scientifically useful?

A feature is useful when it plausibly modifies the response to a randomized price change and is known before the price is shown. Predicting baseline purchase propensity is not the primary objective, although some baseline-predictive features may also be effect modifiers.

High-priority feature families include:

- unmodified technical premium and premium components;
- expected claim cost and uncertainty;
- distance from training support of the tariff model;
- competitor premium, rank, or price gap when legally and operationally available before treatment;
- aggregator versus direct channel;
- new business versus renewal;
- previous premium or insurer;
- coverage choices and deductibles fixed before randomization;
- customer shopping behavior observed before the current randomized quote;
- vehicle and driver risk bands;
- geographic and calendar context;
- tariff version and underwriting pathway.

## 23.2 Avoid post-treatment features

Prohibited examples include:

- clicks or dwell time after seeing the randomized price;
- whether the customer requested a discount after treatment;
- payment plan chosen after price display;
- agent actions triggered by the randomized quote;
- revised coverage selected in response to price;
- final displayed premium when it contains post-randomization manual changes, unless used only for compliance analysis.

Including these can leak treatment consequences into the elasticity score and invalidate policy interpretation.

## 23.3 Absolute price, relative price, and competitive position

The randomized multiplier is relative to the base premium, but elasticity may vary with:

- absolute premium level;
- premium relative to expected claims;
- premium relative to prior insurance;
- premium relative to competitor offers;
- customer budget or vehicle value.

These are effect-modifier candidates. They should not be confused with the randomized dose itself.

## 23.4 Tariff uncertainty

If the fair-price model is noisy, useful pre-treatment diagnostics include:

- prediction interval width;
- ensemble disagreement;
- residual uncertainty from historical backtesting;
- out-of-distribution score;
- distance to nearest training observations;
- tariff component overrides;
- data-quality flags.

Such features can help detect contexts where the quote's competitive position is uncertain and price response may differ. They do not repair nonrandom treatment assignment; randomization remains essential.

## 23.5 Missing values

Missingness can be informative but must be handled consistently:

- distinguish structural absence from extraction failure;
- add missing indicators for important numeric fields;

- group rare and missing categorical levels explicitly;
- fit imputers inside folds;
- monitor missingness by arm and time;
- define production fallback behavior.

Tree libraries can handle missing values natively, but the semantics and leakage risks still require audit.

### 23.6 High-cardinality categorical variables

For variables such as vehicle model, postcode, agent, or tariff cell:

- prefer approved domain groupings;
- use frequency thresholds learned in training folds;
- avoid target encoding unless it is nested and treatment-safe;
- prevent one leaf from representing a handful of side-arm buyers;
- consider hashing or regularized encodings only after simple groupings are exhausted.

### 23.7 Interaction discovery

Tree ensembles discover interactions automatically. For linear models, specify only a defensible small set, such as:

- competitor gap by channel;
- premium band by renewal status;
- tariff uncertainty by out-of-distribution flag;
- customer tenure by previous premium change.

An interaction is a claim that the effect of one variable depends on another. Searching enormous numbers of interactions under weak experimental signal is a major overfitting route.

## 24. Interpretation and diagnostics

### 24.1 What feature importance means here

A feature importance score indicates that the model uses a feature to vary predicted price response. It does not prove a causal effect of the feature itself.

The randomized treatment is causal; the discovered relationship between a customer feature and treatment response can still be distorted by:

- correlated features;
- sampling variation;
- model extrapolation;
- temporal drift;
- latent effect modifiers.

### 24.2 Permutation importance

Permutation importance measures the increase in held-out loss when one feature is randomly shuffled. For elasticity models, compute it on the outer-validation pairwise likelihood, not on training loss.

Shuffle within appropriate blocks when a feature's distribution changes strongly by time or channel. Preserve customer groups when repeated quotes exist.

### 24.3 SHAP values

For pairwise models, SHAP values can explain contributions to

$$\widehat{\log RR}_{tC}(x),$$

which is preferable to explaining the pairwise buyer probability because the latter also contains the assignment offset.

Interpretation:

- a negative H/C SHAP contribution means the feature pushes the model toward stronger sensitivity to a price increase;
- a positive L/C SHAP contribution means the feature pushes the model toward a larger response to a discount.

SHAP is a decomposition of the fitted model, not evidence that changing the feature would change elasticity.

### 24.4 Partial dependence and accumulated local effects

Partial dependence averages predictions after setting a feature to selected values. It can evaluate unrealistic combinations when features are correlated.

Accumulated local effects, or ALE, uses local changes within the observed feature distribution and can be safer under correlation. For either method:

- plot log risk ratio together with both log-change and standard relative-change finite elasticities;
- add buyer-density information;
- show uncertainty across folds or bootstrap fits;
- avoid extrapolating beyond support;
- stratify by major operational context when interactions are strong.

### 24.5 Segment summaries

For governance, create tables with:

- quote and buyer counts;
- side-arm buyer counts;
- predicted mean log risk ratios;
- experimental group risk ratios;
- confidence intervals;
- calibration residuals;
- recommended-arm proportions;
- contribution-value estimates.

Do not publish fine segments with inadequate side-arm counts.

### 24.6 Counterfactual feature misuse

Do not ask the fitted model what would happen if an immutable customer feature were changed and interpret that as a causal effect. The model estimates treatment response conditional on observed context, not causal effects of context variables.

## 25. Common failure modes and dead ends

### 25.1 Learning on buyers and evaluating only on buyers

Buyer likelihood is correct for shape estimation, but the final price policy must be evaluated on all eligible randomized quotes. Otherwise the evaluation conditions on a post-treatment outcome and cannot estimate per-quote business value.

### 25.2 Using nominal propensities when allocation varied

If row-level assignment probabilities changed, using $0.1/0.8/0.1$ everywhere biases the Bayes correction. The exact logged propensities are part of the label-generating process.

### 25.3 Random row splits

Repeated customers can leak nearly identical quotes across folds. Time drift can also be hidden. Use group-disjoint folds and a final time holdout.

### 25.4 Optimizing AUC or VUS

Ranking metrics can reward a model with badly calibrated probability ratios. They are secondary diagnostics only.

## 25.5 Balancing the buyer classes

Balancing removes or shifts part of the treatment signal. Use exact offsets or analytically correct sampling corrections.

## 25.6 Tuning and calibrating on the same held-out rows

Hyperparameter selection, early stopping, and calibration all consume validation information. Nest these operations inside the outer training folds.

## 25.7 Comparing unequal model problems

A strongly structured buyer model should not be declared superior because it beats a generic full-cohort classifier solving a harder baseline-conversion task. Match the price-response parameterization when studying buyer-versus-full efficiency.

## 25.8 Treating a synthetic generator as evidence

Simulation is useful for software checks and controlled sensitivity analysis, but the model that generated the outcomes is privileged by construction. Synthetic rankings do not identify the best real insurance model. Real randomized holdout evidence is required.

## 25.9 Averaging individual elasticities to validate a group effect

A group risk ratio is a ratio of average risks, not the mean of individual log-risk ratios. Use the correct randomized group estimand.

## 25.10 Excessive two-gap flexibility

High and low responses are estimated from limited side-arm buyers. A two-gap model can turn noise into apparent curvature. Require clear out-of-fold likelihood and calibration gains over the scalar model.

## 25.11 Extreme tree predictions

Tiny leaves can create enormous $\left|\log RR\right)$. Strong leaf constraints, regularization, calibration, and tail diagnostics are mandatory. Arbitrary clipping alone hides rather than solves the problem.

## 25.12 Ignoring contribution economics

The most elastic customers are not automatically the best discount targets, and the least elastic are not automatically the best price-increase targets. Decisions depend on both relative demand and the contribution ratios $g_t / g_C$.

## 25.13 Using claim outcomes caused by selection

Changing price can alter the risk mix of buyers. If $g_t(x)$ uses expected claim cost, the insurer must decide whether claim severity among purchasers is assumed invariant or separately modeled. A randomized price policy may select different unobserved risks even within the same observed $X$.

## 25.14 Searching until the final holdout improves

The first look at the final holdout turns it into validation data. Repeatedly revising the model after that destroys the meaning of its performance estimate.

# 26. Model-selection gates

A personalized model should be accepted only when all applicable gates pass.

## Gate 1: experiment validity

- assignment propensities are verified;
- eligibility and outcomes are complete;
- no material post-randomization exclusion or leakage is present;
- repeat quotes are handled;
- treatment compliance is understood.

## Gate 2: stable global signal

- global H/C and L/C effects have sensible uncertainty;
- signs are not driven by one period or channel;
- scalar and two-gap global models are reconciled;
- the outcome window is robust.

## Gate 3: proper likelihood improvement

- flexible models beat the global anchor out of fold;
- paired confidence intervals support the improvement;
- gains appear in both development backtests and the final temporal holdout;
- one contrast does not conceal failure in the other.

## Gate 4: calibration

- calibration intercept is near zero;
- slope is positive and not near zero;
- grouped randomized effects move consistently with predictions;
- projection and tail behavior are acceptable.

## Gate 5: stability

- rankings persist across folds, seeds, periods, and reasonable preprocessing choices;
- top-target overlap is sufficient for operations;
- feature explanations are plausible and not dominated by leakage proxies.

## Gate 6: business value

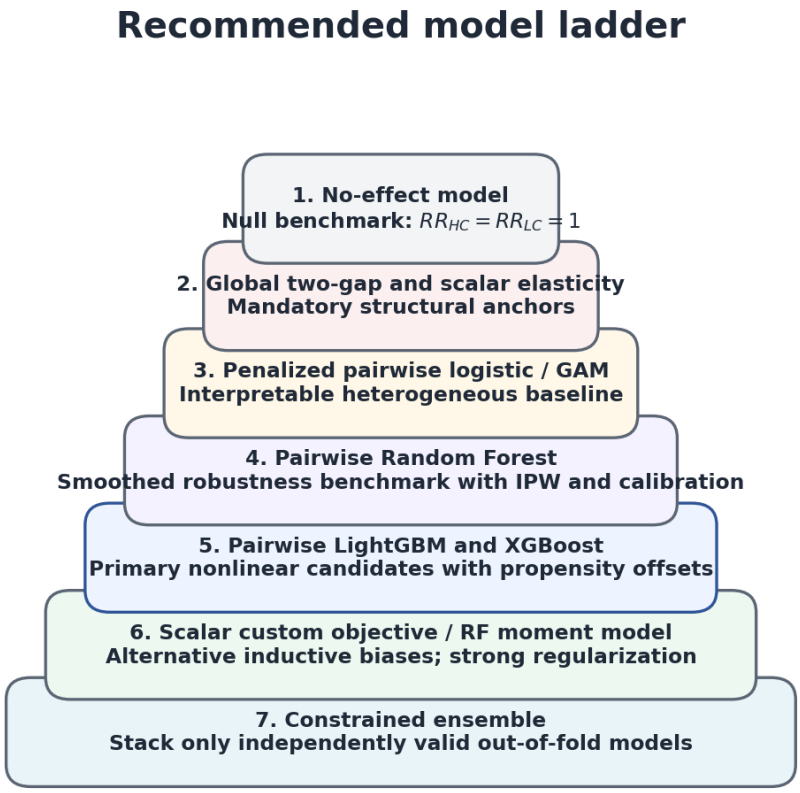
- out-of-fold IPS or DR value improves over uniform and global-elasticity policies;
- the lower confidence bound is acceptable;
- value survives transaction costs, policy constraints, and abstention rules.

## Gate 7: governance and deployment

- features are legally and operationally approved;
- model versioning and logging are complete;
- monitoring and rollback are defined;
- a continuing randomized exploration plan protects against drift and feedback.

When a gate fails, the correct response may be to deploy the global model or retain control—not to increase model complexity.

# 27. Recommended practical sequence




*A complex model is eligible only after it beats the simpler anchor out of fold, remains calibrated, and improves randomized business value.*

*Recommended model ladder*

1. Audit the randomization and data-generating process.
2. Estimate model-free arm risks and risk ratios.
3. Fit no-effect, global two-gap, and global scalar anchors.
4. Fit penalized pairwise logistic models.
5. Fit pairwise LightGBM and XGBoost with exact offsets.
6. Fit a strongly smoothed IPW Random Forest benchmark.
7. Fit scalar LightGBM and XGBoost custom-objective challengers.
8. Produce inner out-of-fold predictions and fit affine calibrators.
9. Reconstruct coherent three-arm buyer posteriors.
10. Evaluate proper likelihood, calibration, grouped randomized effects, stability, and policy value.
11. Consider a constrained ensemble only among independently valid components.
12. Evaluate the frozen final system once on the chronological holdout.
13. Begin with shadow deployment and continuing randomized exploration.

# 28. Constrained ensembles

## 28.1 Why ensemble log risk ratios

Suppose $K$ valid models produce high-arm log risk ratios $\ell_{H,k}(x) \le 0$. A convex ensemble is

$$\ell_H^{ens}(x) = \sum_{k=1}^{K} w_k \ell_{H,k}(x),$$

where

$$w_k \geq 0, \sum_k w_k = 1.$$

Then $\ell_H^{ens}(x) \leq 0$. The same argument preserves $\ell_L^{ens}(x) \geq 0$.

This is preferable to averaging probabilities from models that used different propensity handling.

## 28.2 Learning ensemble weights

Use inner out-of-fold log-risk-ratio predictions and solve

$$\min_{w \in \Delta^{K-1}} NLL\left(\sum_k w_k \ell_k\right),$$

where $\Delta^{K-1}$ is the weight simplex.

Fit separate weights for H/C and L/C unless a strong reason supports shared weights. Then recalibrate the ensemble out of fold.

## 28.3 Diversity requirements

Do not add a model merely because its algorithm name is different. A useful component should:

- have acceptable standalone likelihood and calibration;
- make meaningfully different residual errors;
- remain stable across folds;
- improve ensemble out-of-fold loss after accounting for complexity.

# 29. Production monitoring

## 29.1 What must be logged

For every scored quote, store:

- model and calibration version;
- immutable feature snapshot or hash;
- predicted $\log RR_{HC}$ and $\log RR_{LC}$;
- finite-difference elasticities;
- contribution values $g_H, g_C, g_L$;
- decision scores $S_H, S_L$;
- recommended and actual arm;
- randomization propensities when exploration continues;
- reason codes and abstention status;
- eventual purchase and outcome timestamp.

## 29.2 Drift monitoring

Monitor:

- feature distribution drift;
- missingness drift;
- score distribution drift;
- recommended-arm proportions;
- propensity and assignment drift;
- buyer arm composition;
- randomized group calibration;
- realized contribution value;
- performance by tariff version, channel, and time.

Score drift alone does not prove performance drift, but it identifies where randomized revalidation should concentrate.

### 29.3 Continued exploration

A deterministic learned policy eventually removes overlap and makes future causal evaluation difficult. Maintain controlled randomized exploration, especially:

- near decision boundaries;
- in new tariff versions or channels;
- in sparse customer regions;
- when model disagreement is high;
- when market competition changes.

The exploration probabilities must be logged and remain positive for the arms that future evaluation may compare.

## 30. Compact implementation blueprint

```
# Pseudocode: the orchestration, not a drop-in one-file production system.

def run_elasticity_pipeline(data, feature_manifest, config):
    # 1. Validate and audit.
    validate_experiment_frame(data)
    features = approve_pre_treatment_features(data, feature_manifest)
    audit = audit_randomization_and_outcomes(data, features, config)
    audit.raise_if_blocking_findings()

    # 2. Freeze temporal and grouped splits.
    development, final_holdout = chronological_split(data, config.holdout_start)
    outer_folds = make_group_disjoint_outer_folds(
        development,
        group_col=config.group_col,
        arm_col="arm",
    )

    oof_predictions = []

    # 3. Outer cross-fitting.
    for outer_train_idx, outer_valid_idx in outer_folds:
        train = development.iloc[outer_train_idx]
        valid = development.iloc[outer_valid_idx]

        train_buyers = train[train.purchase == 1]
        valid_buyers = valid[valid.purchase == 1]

        # Mandatory fold-local global anchors.
        global_scalar = fit_global_scalar_on_buyers(train_buyers)
        global_pair = fit_global_two_gap_on_buyers(train_buyers)

        for family in ["logistic", "lightgbm", "xgboost", "random_forest"]:
            for contrast in ["H_vs_C", "L_vs_C"]:
                study = tune_with_optuna_nested_cv(
                    buyers=train_buyers,
                    features=features,
                    family=family,
                    contrast=contrast,
                    config=config,
                )

                # Inner OOF predictions for calibration.
                raw_inner_oof = cross_fit_selected_model(
                    train_buyers,
                    features,
                    family,
                    contrast,
                    study.best_params,
```

```
                config,
            )
            calibrator = fit_exact_offset_calibrator(raw_inner_oof)

            # Refit on all outer training buyers and score outer validation.
            base_model = fit_selected_model(
                train_buyers,
                features,
                family,
                contrast,
                study.best_params,
                config,
            )
            raw_valid = base_model.predict_log_rr(valid[features])
            calibrated_valid = calibrator.predict_log_rr(raw_valid)

            save_outer_predictions(
                valid,
                family,
                contrast,
                calibrated_valid,
                global_scalar,
                global_pair,
                study,
            )

# 4. Compare complete systems on development OOF predictions.
results = evaluate_elasticity_systems(
    development,
    oof_predictions,
    cluster_col=config.group_col,
    contribution_columns=config.contribution_columns,
)
selected_specification = apply_predeclared_selection_gates(results, config)

# 5. Freeze, refit on all development data, and score holdout once.
final_model = refit_frozen_specification(
    development,
    selected_specification,
    features,
    config,
)
holdout_predictions = final_model.predict(final_holdout[features])
final_report = evaluate_frozen_holdout(
    final_holdout,
    holdout_predictions,
    selected_specification,
    config,
)
return final_model, results, final_report
```

The helper names intentionally expose the methodological stages. A production implementation should separate immutable data extraction, audit, fitting, evaluation, artifact storage, and deployment code.

## 31. Notation glossary

| Symbol | Name | Meaning |
|---|---|---|
| $i$ | row index | One eligible quote opportunity |
| $x$ or $X_i$ | feature context | Pre-treatment customer, vehicle, quote, and market information |
| $T_i$ | treatment arm | Randomized arm $H$, $C$, or $L$ |
| $H, C, L$ | arm labels | High price, control price, low price |
| $Y_i$ | observed outcome | 1 for purchase within the outcome window, 0 otherwise |
| $Y_i(t)$ | potential outcome | Purchase that would occur for quote |

| Symbol | Name | Meaning |
|---|---|---|
| | | $i$ under arm $t$ |
| $\pi_{i,t}$ | propensity | Probability that the randomizer assigns arm $t$ to quote $i$ |
| $q_t(x)$ | arm conversion risk | $P(Y=1 \mid T=t, X=x)$ |
| $r_t(x)$ | buyer-arm posterior | $P(T=t \mid Y=1, X=x)$ |
| $RR_{tC}(x)$ | risk ratio | $q_t(x)/q_C(x)$ |
| $\ell_t(x)$ | log risk ratio | $\log RR_{tC}(x)$ |
| $P_t$ | premium | Premium under arm $t$ |
| $m_{tC}$ | price ratio | $P_t/P_C$ |
| $\delta_{tC}$ | arithmetic relative price change | $m_{tC}-1$ |
| $z_t$ | log-price dose | Usually $\log(P_t/P_C)$ |
| $d_t$ | centered log dose | $z_t - z_C$; often equal to $z_t$ when $z_C=0$ |
| $\varepsilon_{tC}^{rel}$ | standard finite elasticity | $(RR_{tC}-1)/(m_{tC}-1)$, relative to control |
| $\varepsilon_{tC}^{\log}$ | log-change finite elasticity | $\log RR_{tC}/\log m_{tC}$ |
| $\varepsilon_{tC}^{mid}$ | midpoint arc elasticity | Symmetric finite-change convention using midpoint denominators |
| $\beta^{rel}, \beta^{\log}$ | positive sensitivity magnitudes | Negative of the corresponding signed elasticity |
| $g_t(x)$ | contribution on sale | Economic contribution conditional on purchase under arm $t$ |
| $V_t(x)$ | expected contribution | $q_t(x)g_t(x)$ per eligible quote |
| $d(x)$ | pricing policy | Function selecting $H$, $C$, or $L$ |
| $S_t(x)$ | relative contribution score | $\log(g_t/g_C)+\log RR_{tC}$ |
| $\sigma(u)$ | logistic function | $1/(1+e^{-u})$ |
| $logit(p)$ | log odds | $\log[p/(1-p)]$ |
| `softmax` | multiclass normalization | Converts a score vector to probabilities summing to one |
| `softplus` | positive transform | $\log(1+e^{u})$ |
| NLL | negative log-likelihood | Proper loss used for model fitting and comparison |
| IPW/IPS | inverse propensity weighting/scoring | Weighting by reciprocal assignment probability |
| OOF | out of fold | Prediction made by a model not trained on that row |
| CV | cross-validation | Repeated train/validation splitting |
| TPE | Tree-structured Parzen Estimator | Optuna's adaptive hyperparameter sampler |
| LCB | lower confidence bound | Conservative lower endpoint for a value or gain |

# 32. Derivation summary

## 32.1 Bayes flip

$$r_t(x)=\frac{\pi_t(x)\,q_t(x)}{\sum_j \pi_j(x)\,q_j(x)}.$$

## 32.2 Relative risk recovery

$$RR_{tC}(x)=\frac{r_t(x)/\pi_t(x)}{r_C(x)/\pi_C(x)}.$$

## 32.3 Pairwise offset

$$\operatorname{logit} P(T=t \mid T\in\{t,C\}, Y=1, X=x)=\log\frac{\pi_t(x)}{\pi_C(x)}+\log RR_{tC}(x).$$

## 32.4 Standard relative-change elasticity

$$\varepsilon_{tC}^{rel}(x)=\frac{RR_{tC}(x)-1}{P_t/P_C-1}.$$

## 32.5 Log-change finite elasticity

$$\varepsilon_{tC}^{\log}(x)=\frac{\log RR_{tC}(x)}{\log(P_t/P_C)}.$$

## 32.6 Conversion between conventions

$$\varepsilon^{\log}=\frac{\log\left[1+\varepsilon^{rel}(m-1)\right)}{\log m}, \varepsilon^{rel}=\frac{m^{\varepsilon^{\log}}-1}{m-1}.$$

## 32.7 Log-constant scalar model

$$RR_{tC}(x)=\exp\left[-\beta^{\log}(x)\log(P_t/P_C)\right).$$

## 32.8 Constant standard-relative scalar model

$$RR_{tC}(x)=1-\beta^{rel}(x)(P_t/P_C-1).$$

## 32.9 Business arm choice

$$d^{i}(x)=\arg\max_t g_t(x)\,RR_{tC}(x).$$

## 32.10 Pairwise likelihood

$$L_i=\log(1+e^{o_i+\ell_i})-Z_i(o_i+\ell_i).$$

## 32.11 IPS policy value

$$\hat{V}_{IPS}(d)=\frac{1}{N}\sum_i \frac{1\{T_i=d(X_i)\}\,Y_i\,g_{T_i}(X_i)}{\pi_{i,T_i}}.$$

# 33. Hyperparameter reference tables

## 33.1 Pairwise LightGBM

| Hyperparameter | Suggested search | Main role |
|---|---|---|
| `learning_rate` | 0.005–0.10, log | Boosting step size |
| `max_depth` | 2–7 | Interaction depth |
| `num_leaves` | 4–$min\left(64, 2^d - 1\right)$ | Tree complexity |
| `min_data_in_leaf` | dynamic, log | Prevent sparse elasticity regions |
| `min_sum_hessian_in_leaf` | 0.1–50, log | Minimum local information |
| `feature_fraction` | 0.5–1.0 | Feature subsampling |
| `bagging_fraction` | 0.6–1.0 | Row subsampling |
| `bagging_freq` | 1–10 | Resampling frequency |
| `lambda_l1` | $10^{-8}$–100, log | L1 leaf regularization |
| `lambda_l2` | $10^{-3}$–100, log | L2 leaf regularization |
| `min_gain_to_split` | 0 or $10^{-5}$–1 | Split threshold |
| `max_bin` | 63, 127, 255, 511 | Histogram resolution |

## 33.2 Pairwise XGBoost

| Hyperparameter | Suggested search | Main role |
|---|---|---|
| `eta` | 0.005–0.10, log | Boosting step size |
| `grow_policy` | depthwise/lossguide | Tree-growth rule |
| `max_depth` | 2–7 | Depthwise complexity |
| `max_leaves` | 8–64 | Lossguide complexity |
| `min_child_weight` | 1–100, log | Minimum child information |
| `subsample` | 0.6–1.0 | Row subsampling |
| `colsample_bytree` | 0.5–1.0 | Feature subsampling |
| `gamma` | 0 or $10^{-6}$–10 | Split threshold |
| `reg_alpha` | $10^{-8}$–100, log | L1 leaf regularization |
| `reg_lambda` | $10^{-3}$–100, log | L2 leaf regularization |
| `max_delta_step` | 0, 1, 2, 5 | Update-size cap |
| `max_bin` | 64, 128, 256, 512 | Histogram resolution |
| `scale_pos_weight` | fixed at 1 | Preserve posterior target |

## 33.3 Pairwise Random Forest

| Hyperparameter | Suggested search | Main role |
|---|---|---|
| `n_estimators` | fixed near 1000 | Monte Carlo stability |
| `criterion` | log_loss/gini | Split criterion |
| `max_depth` | 4–16 | Tree depth |
| `min_samples_leaf` | dynamic | Primary smoothing control |
| `max_features` | sqrt/log2/0.2–1.0 | Tree diversity |
| `max_samples` | 0.5–1.0 | Bootstrap sample fraction |
| `min_impurity_decrease` | 0 or $10^{-8}$–$10^{-3}$ | Split threshold |
| `ccp_alpha` | 0 or $10^{-8}$–$10^{-2}$ | Cost-complexity pruning |
| `class_weight` | fixed `None` | Preserve weighted estimand |

# 34. Final checklist for an LLM agent

Before reporting any model as successful, the agent must answer all of the following with artifacts, not assertions:

1. What exact randomized estimand is being modeled?
2. What are the logged row-level propensities, and how were they verified?
3. Which variables are known before treatment?
4. How are repeat customers grouped?
5. What is the final chronological holdout period?
6. What are the buyer counts in H, C, and L for every fold?
7. What are the no-effect, global two-gap, and global scalar results?
8. How are offsets or inverse-propensity weights implemented?
9. How is class balancing prevented or corrected?
10. What are the exact Optuna search spaces and trial budgets?
11. How is early stopping nested?
12. How are calibration predictions generated out of fold?
13. What are the calibration intercept and slope?
14. What is the information gain over the global anchor, with a paired interval?
15. Do grouped randomized effects agree with predicted ordering and magnitude?
16. How stable are rankings across time, seeds, and folds?
17. What fraction of predictions violate monotonicity before projection?
18. What are the tail distributions of log risk ratio and elasticity? Which elasticity convention and reference direction are reported?
19. What is the out-of-fold IPS or DR value relative to uniform and global policies?
20. Does the lower confidence bound support deployment?
21. What regulatory, fairness, and operational restrictions apply?
22. How will continued randomized exploration be maintained?
23. Which findings are empirical, which are algebraic, and which are assumptions?
24. What unfavorable or null findings were retained in the report?

## 35. References and further reading


1. Dai, J. Y., LeBlanc, M., and Kooperberg, C. “Case-only approach to identifying markers predicting treatment effects on the relative risk scale.” *Biometrics* 74(2), 2018. This establishes the randomized case-only relative-risk logic used by the Bayes-flip formulation.
2. Gneiting, T., and Raftery, A. E. “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association* 102(477), 2007. This explains why log loss evaluates calibrated probabilities rather than rankings alone.
3. Xu, Y., and Yadlowsky, S. “Calibration Error for Heterogeneous Treatment Effects.” *Proceedings of AISTATS*, 2022. This motivates separate evaluation of treatment-effect calibration and ranking.
4. Dudík, M., Langford, J., and Li, L. “Doubly Robust Policy Evaluation and Learning.” *Proceedings of ICML*, 2011. This provides the basis for optional doubly robust policy-value evaluation.
5. scikit-learn documentation: `StratifiedGroupKFold`, `RandomForestClassifier`, preprocessing pipelines, and probability calibration.
6. LightGBM documentation: training parameters, `init_score`, custom objectives, raw scores, and parameter tuning.
7. XGBoost documentation: `base_margin`, custom objectives, prediction margins, histogram trees, and early-stopping prediction ranges.
8. Optuna documentation: studies, TPE sampler, pruning, and reproducible storage.


Official documentation and source links should be version-pinned in the model repository used for the real analysis. Software interfaces change; the statistical estimand should not.

## Closing perspective

The Bayes-flip approach does not make price elasticity easy. It makes the target cleaner.

By conditioning on buyers and using the known randomized assignment probabilities, it directly identifies relative conversion risks—the quantities needed to compare tested price adjustments. Its strongest implementations are not generic class predictors. They are models whose likelihood, offsets, constraints, calibration, validation, and business decision rule all preserve that exact interpretation.

The practical hierarchy is deliberately conservative:

- trust the experiment before the algorithm;
- trust a stable global effect before individualized heterogeneity;
- use proper likelihood before ranking metrics;
- calibrate before acting on magnitude;
- evaluate the policy on all randomized quotes;
- keep a price unchanged when evidence is weak;
- maintain exploration so the model can continue to be tested.

The best model is not the one with the most sophisticated architecture. It is the simplest model that demonstrates reproducible, calibrated, economically valuable heterogeneity on untouched real randomized data.